

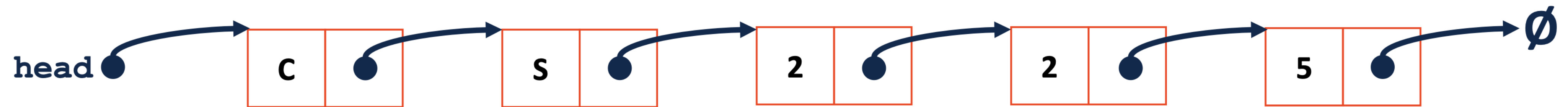
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Linked Memory



Last Lecture :
 { Insert at the front
 { Print in Reverse

What if we want to remove sth. from the list?

- find it first

- delete it

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4
5
6
7
8
9
10 }
```

→ Return ListNode pointer by Reference? Alias of
List.cpp
Some memory address

How to implement : { Recursion
Iteration.

Return ListNode pointer by Reference? Alias of List.cpp

Some memory address

```

1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4
5     ListNode * thru = head_; thru_ is a pointer on the
6
7     stack, so I can change it
8
9
10 }

```

now

How to implement : { Recursion
Iteration.

Return ListNode pointer by Reference? Alias of List.cpp

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4
5     ListNode * thru = head_;
6
7     for (int i = 0; i < index; i++)
8         thru = thru->next;
9
10 }
```

Some memory address

thru_ is a pointer on the stack, so I can change it now

return thru;

How to implement: { Recursion
Iteration.

Good Enough??

Return ListNode pointer by Reference? Alias of List.cpp

Some memory address

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4
5     ListNode * thru = head_;
6
7     for (int i = 0; i < index; i++)
8         thru = thru->next;
9
10 }
```

thru_ is a pointer on the stack, so I can change it now

return thru;

How to implement: { Recursion
Iteration.

~~Good Not Enough?~~

Return a stack variable by reference Bad

Return ListNode pointer by Reference? Alias of List.cpp

Some memory address

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4
5     ListNode * thru = head_; thru_ is a pointer on the
6
7     for (int i = 0; i < index; i++) stack, so I can change it
8
9     thru = thru->next; now
10 }
```

return ~~thru~~ ;

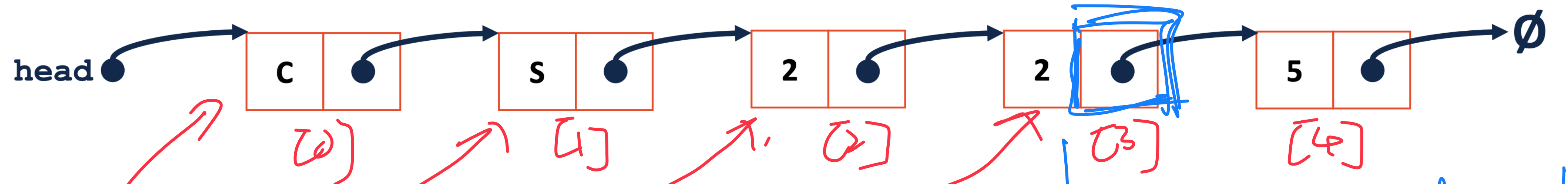
thru->next;

How to implement : { Recursion in the heap.
Iteration.

Need Error checking

Linked Memory

find - (3) ?

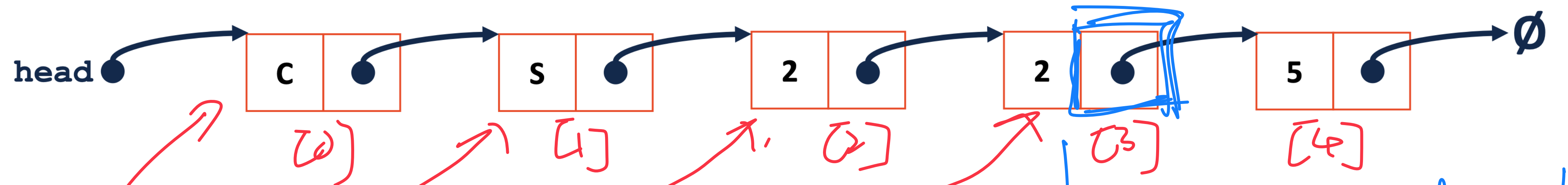


return the address here

find - (3) → data

Linked Memory

find - (3) ?



return the address here

find - (3) → data

?? How to fix ??

Sentinel Node

Specially designed nodes that **do not hold or refer to any data** of the list.

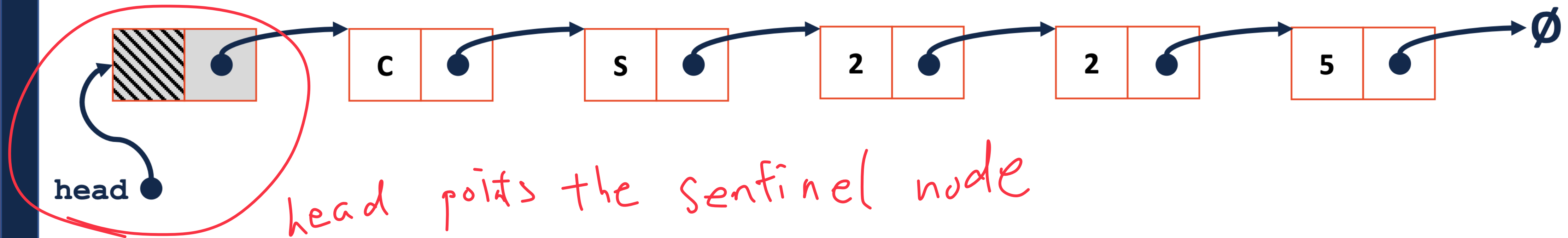
Why?

We do not need to write the different conditions for every operation.
Programming easier.

E.g., different conditions of insertion:

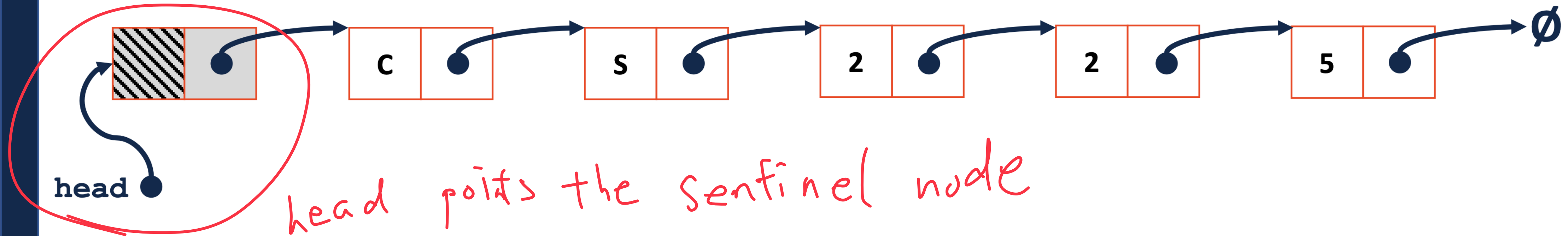
- At the front of the linked list
- After a given node.
- At the end of the linked list.

Sentinel Node



head points the sentinel node
Empty List: head $\rightarrow$ next = Null

Sentinel Node

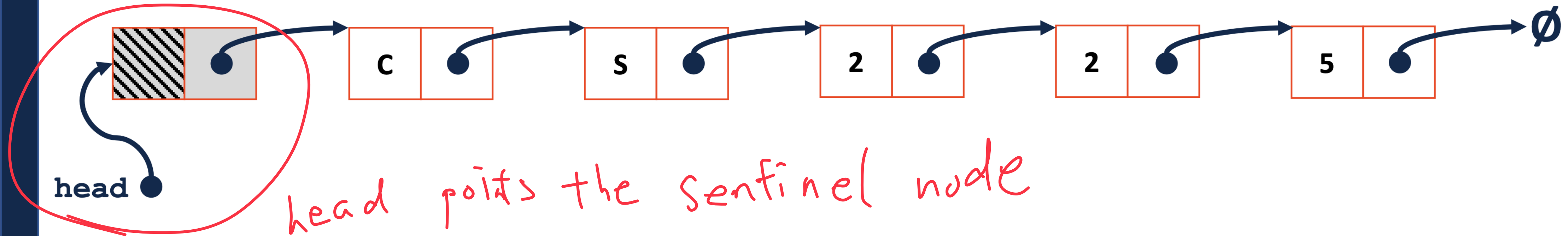


head points the sentinel node

Empty List: head $\rightarrow$ next = Null

Overhead { storage : 1^n
run-time : 1^n

Sentinel Node

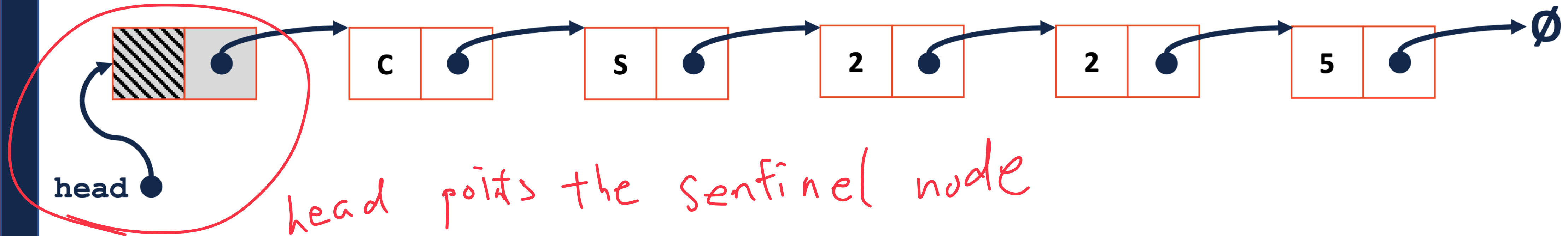


head points the sentinel node

Empty List: head $\rightarrow$ next = Null

Overhead { Storage : $O(1)$
run-time:

Sentinel Node



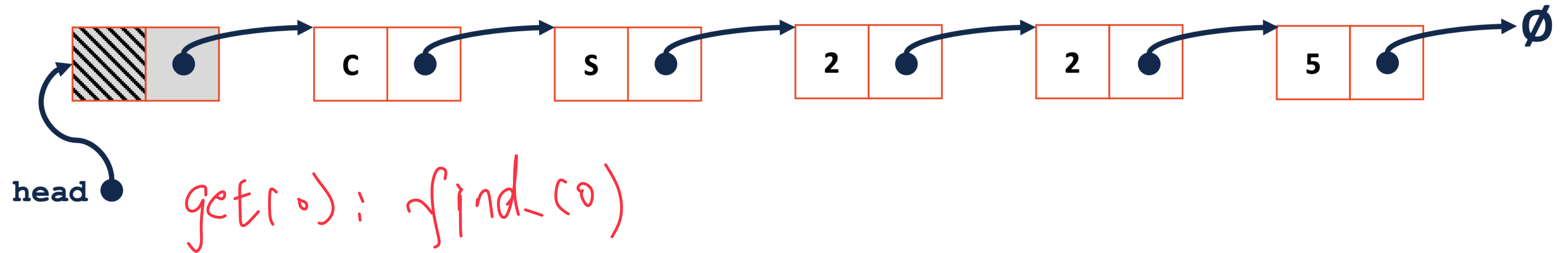
head points the sentinel node

Empty List: head $\rightarrow$ next = Null

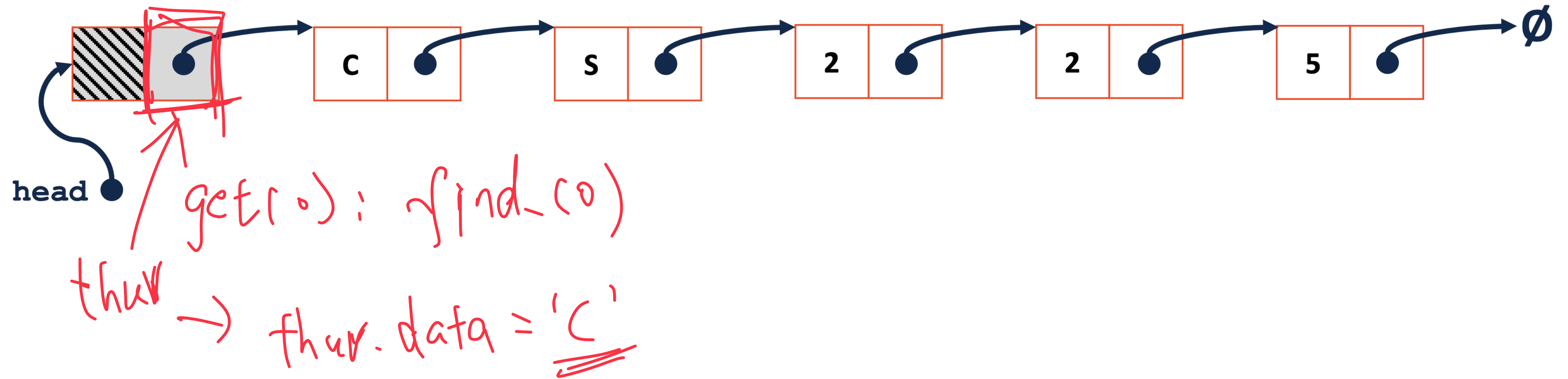
Overhead { storage : $O(1)$
run-time: $O(1)$

```
1  #include "List.h"
2
3  ListNode *& List::find_(unsigned index) const {
4      ListNode *thru = head;
5      for (unsigned i=0; i < index; i++) {
6          thru = thru->next;
7      }
8      return thru->next;
9  }
10
11 T & List::get(unsigned index) const {
12     ListNode* & ptr = find_(index);
13
14     return ptr->data;
15
16 }
17
18 }
```

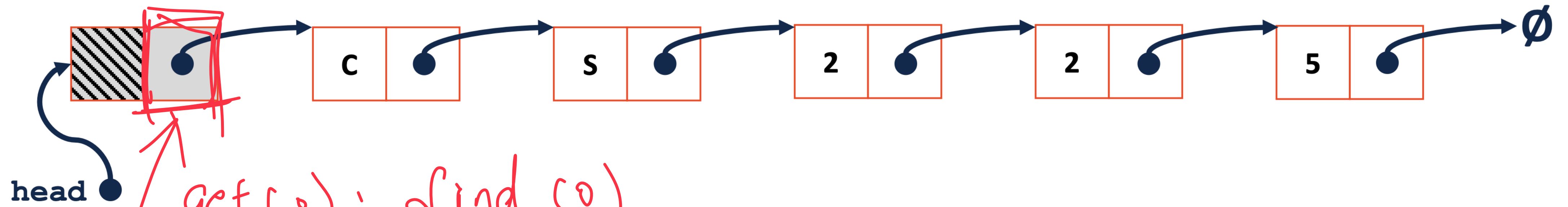

Sentinel Node



Sentinel Node



Sentinel Node

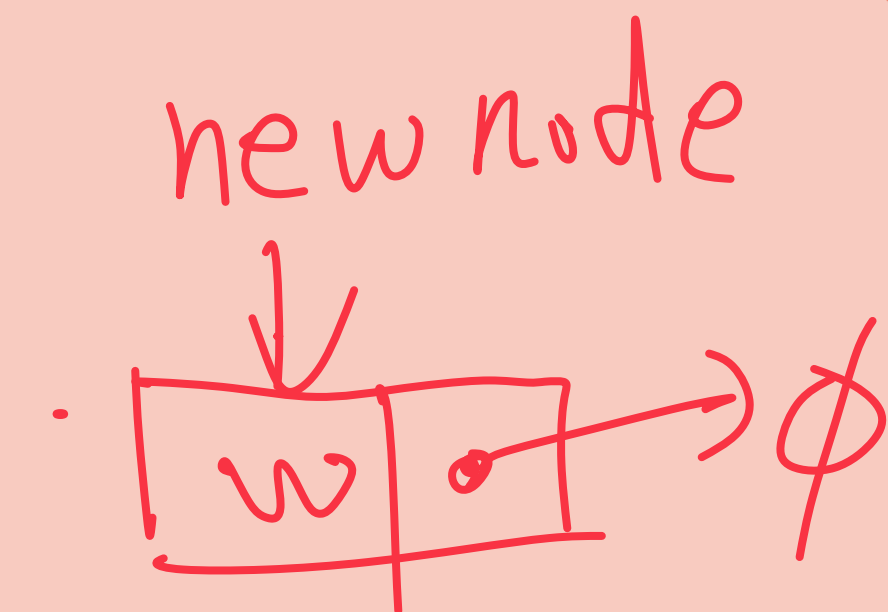


get() : find co
thru $\rightarrow$ thru.data = 'c'

- No need to check for NULL of head.

List.cpp

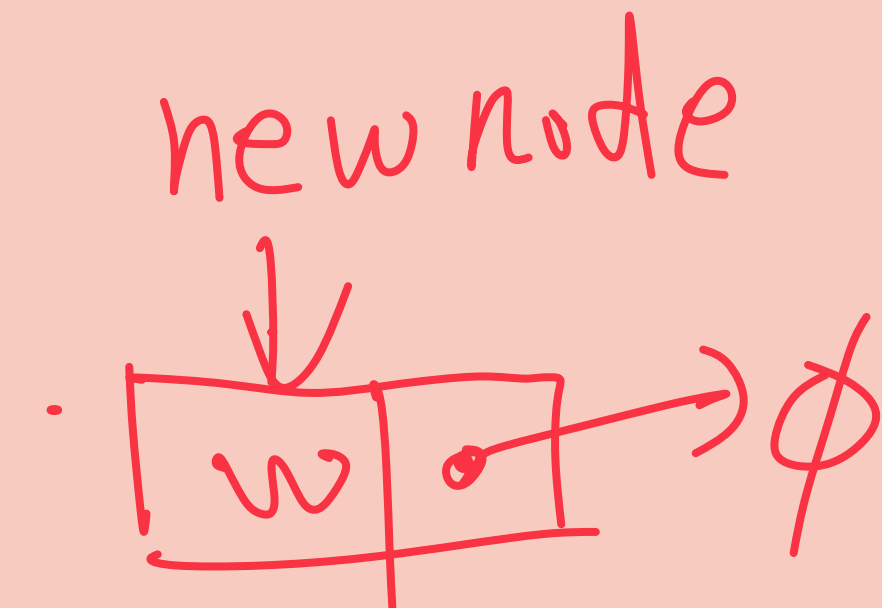
```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     ListNode *thru = head;
5     for (unsigned i=0; i < index; i++) {
6         thru = thru->next;
7     }
8     return thru->next;
9 }
10
11 void List::insert(T & t, unsigned index) {
12
13
14
15
16
17
18 }
```



$\text{ListNode } * \text{newnode} = \text{new ListNode}(t);$

List.cpp

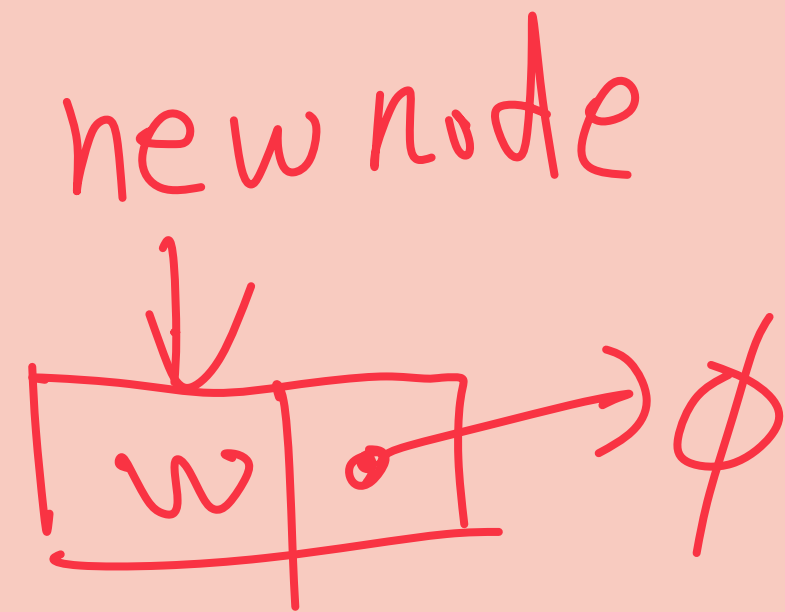
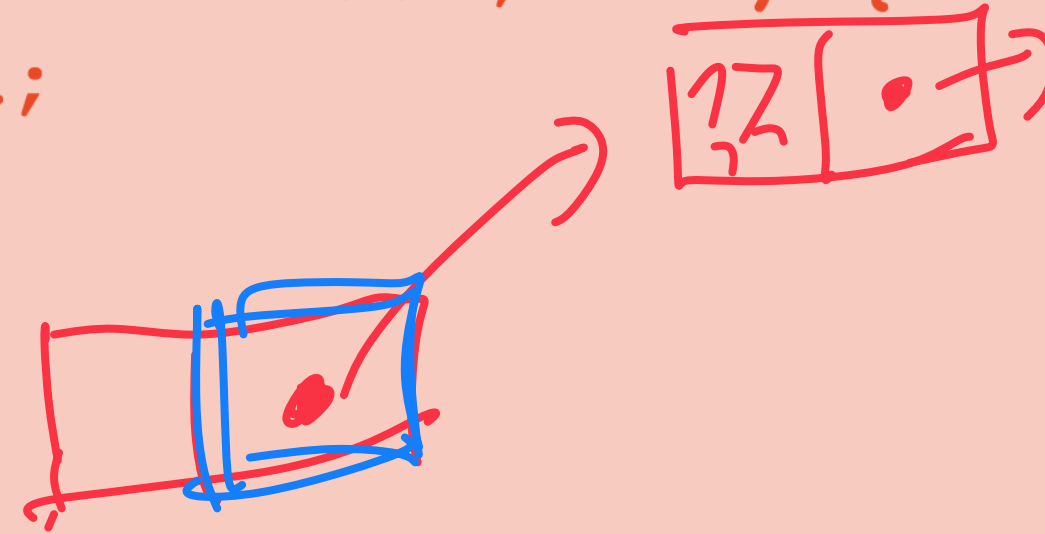
```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     ListNode *thru = head;
5     for (unsigned i=0; i < index; i++) {
6         thru = thru->next;
7     }
8     return thru->next;
9 }
```



```
11 void List::insert(T & t, unsigned index) {
12     ListNode *newnode = new ListNode(t);
13
14     ListNode *pfr = find_(index);
15
16
17
18 }
```


List.cpp

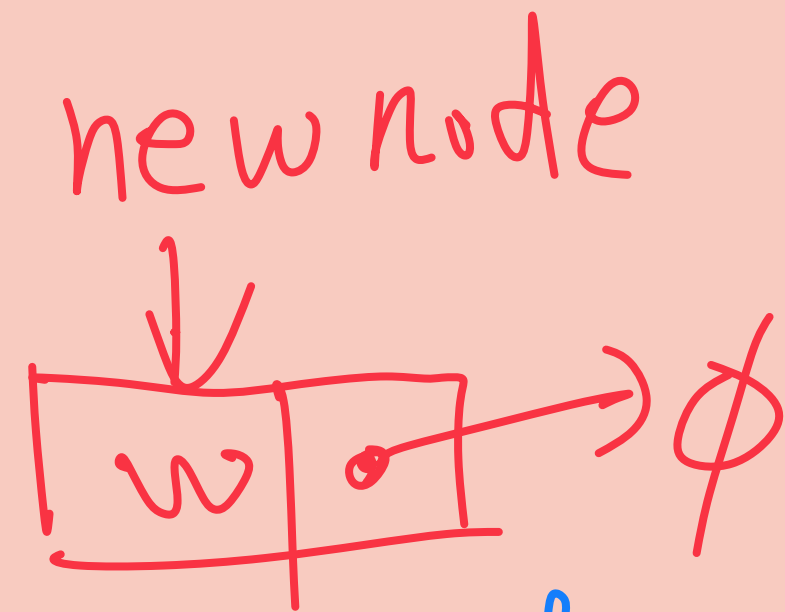
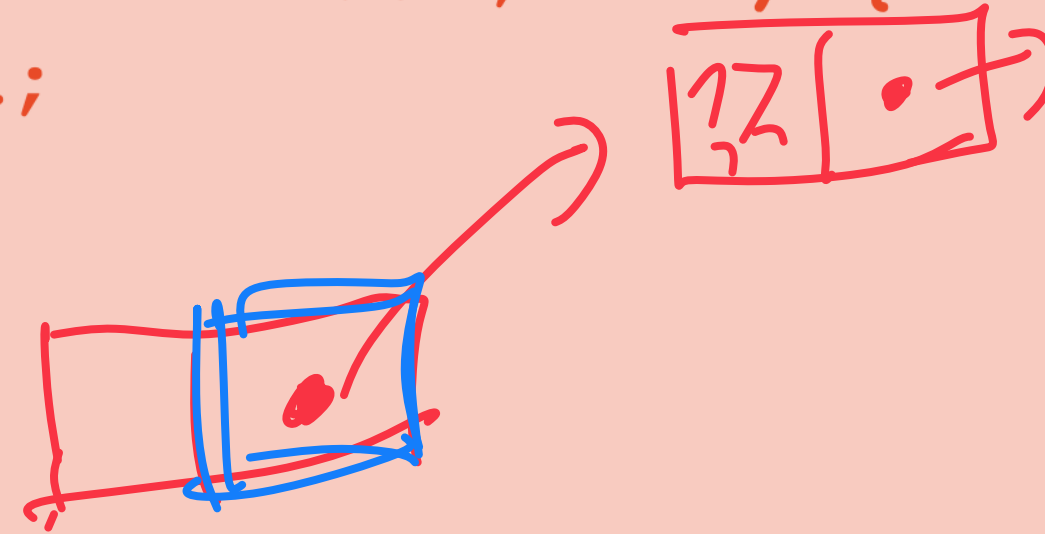
```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     ListNode *thru = head;
5     for (unsigned i=0; i < index; i++) {
6         thru = thru->next;
7     }
8     return thru->next;
9 }
```



```
11 void List::insert(T & t, unsigned index) {
12     ListNode *newnode = new ListNode(t);
13
14     ListNode *pfor = find_(index);
15
16
17
18 }
```

List.cpp

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     ListNode *thru = head;
5     for (unsigned i=0; i < index; i++) {
6         thru = thru->next;
7     }
8     return thru->next;
9 }
```



```
11 void List::insert(T & t, unsigned index) {
```

12 $\text{ListNode } * \text{newnode} = \text{new ListNode}(t);$ How??

13 $\text{ListNode } * \text{pfr} = \text{find_}(index);$

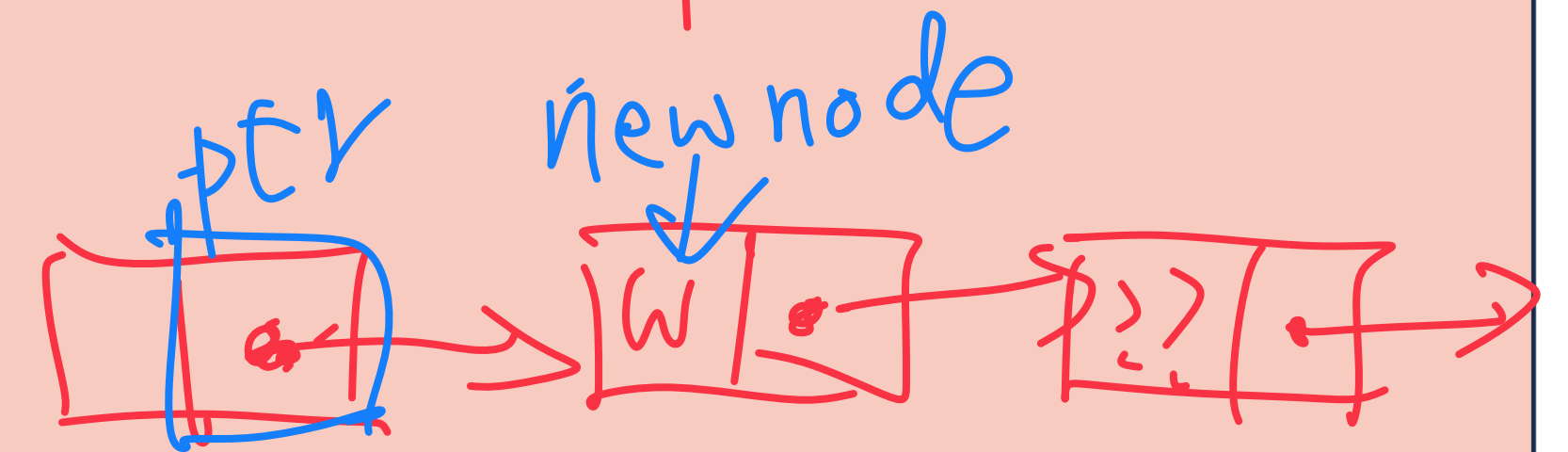
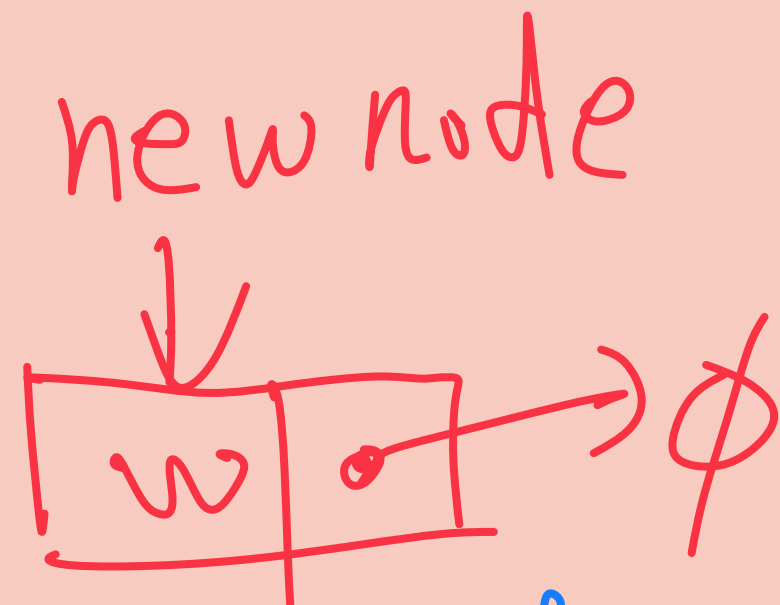
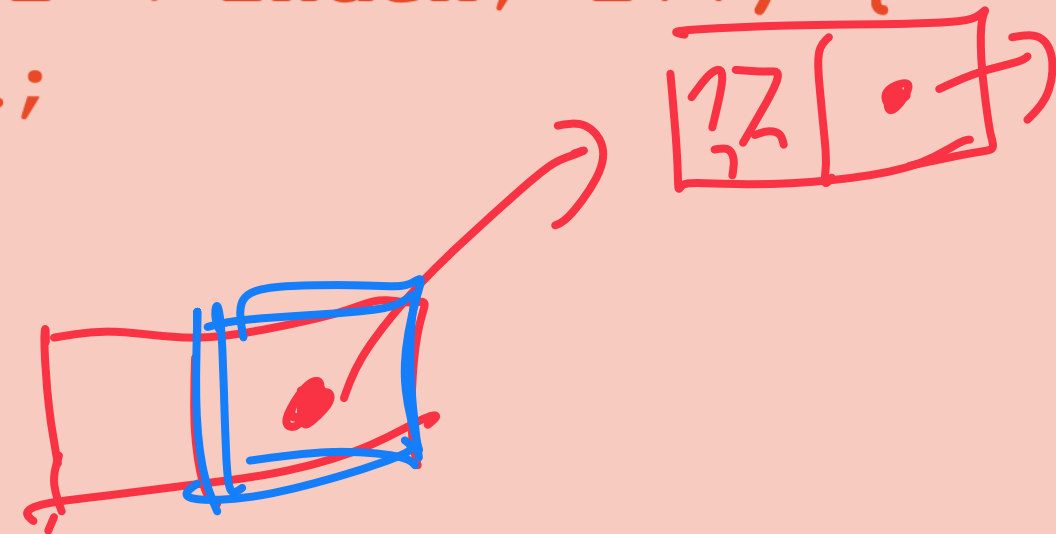
```
14
15
16
17
18 }
```

List.cpp

```

1  #include "List.h"
2
3  ListNode *& List::find_(unsigned index) const {
4      ListNode *thru = head;
5      for (unsigned i=0; i < index; i++) {
6          thru = thru->next;
7      }
8      return thru->next;
9  }

```



```

11 void List::insert(T & t, unsigned index) {
12
13     ListNode *newnode = new ListNode(t);
14
15     ListNode *ptr = find_(index);
16
17     newnode->next = ptr;
18 }

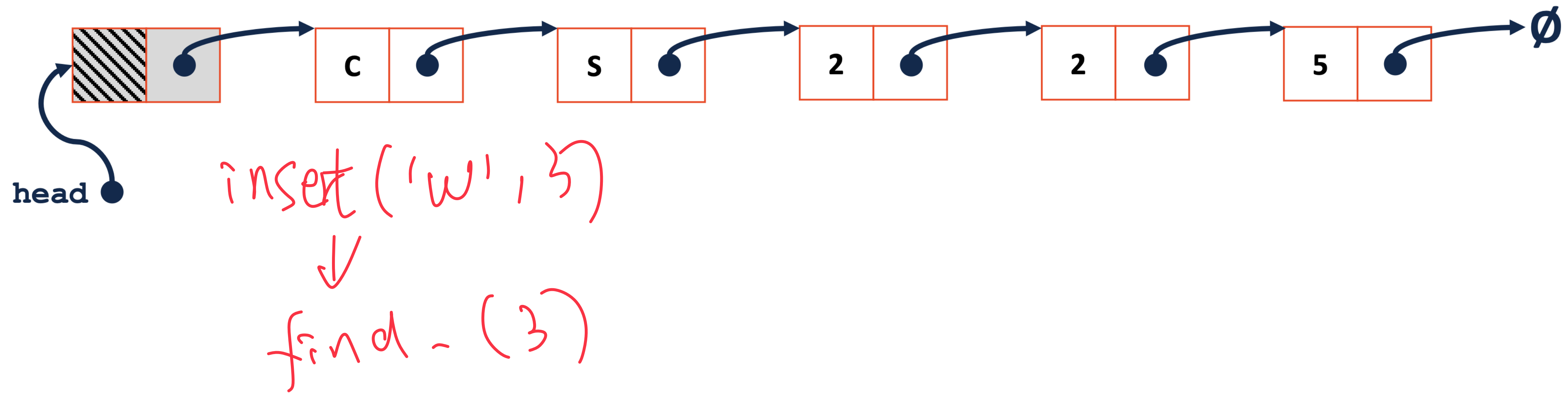
```

How ??

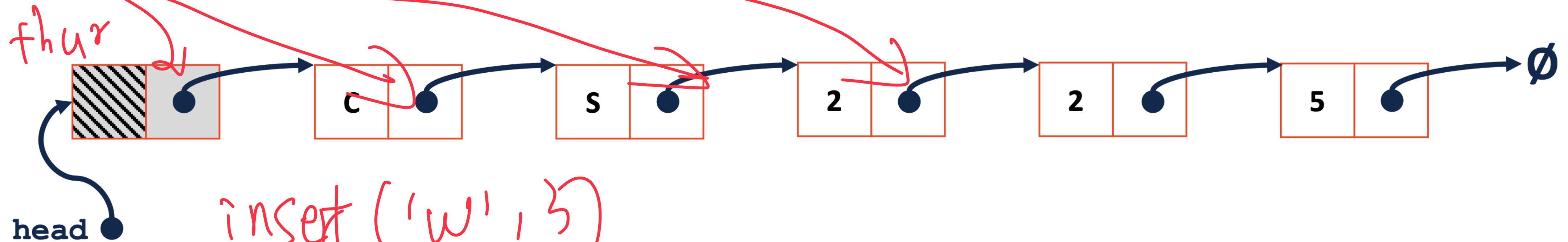
ptr = newnode;

ptr = newnode;

Sentinel Node



Sentinel Node

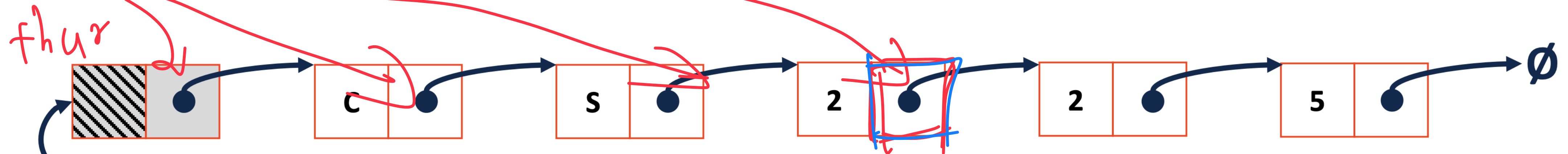


insert('w', 3)

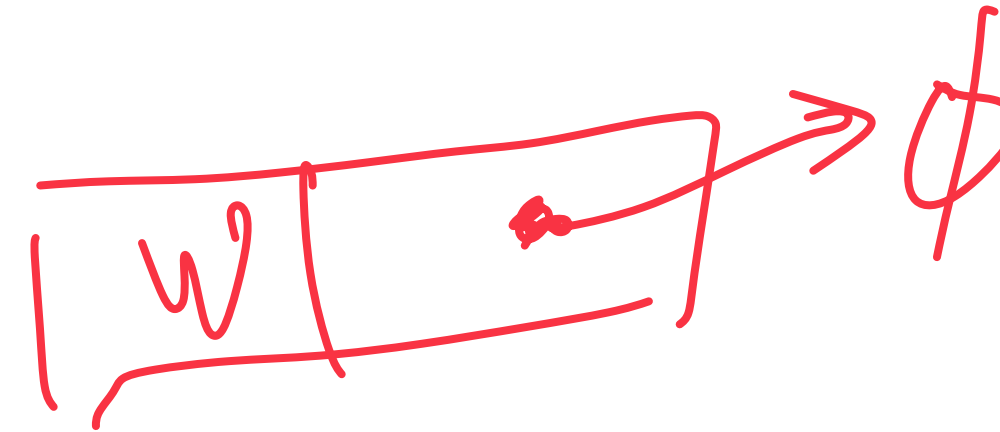


find - (3)

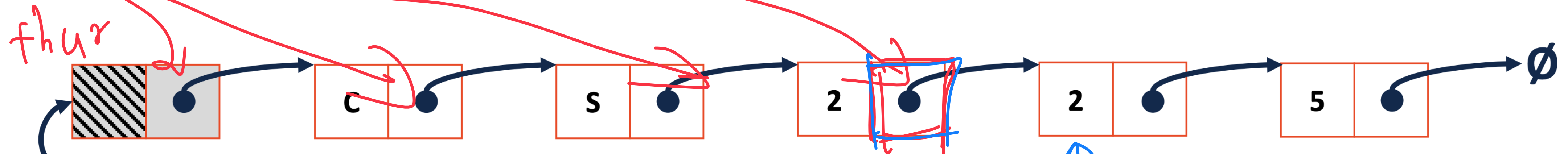
Sentinel Node



insert('w', 3)
↓
find - (3)



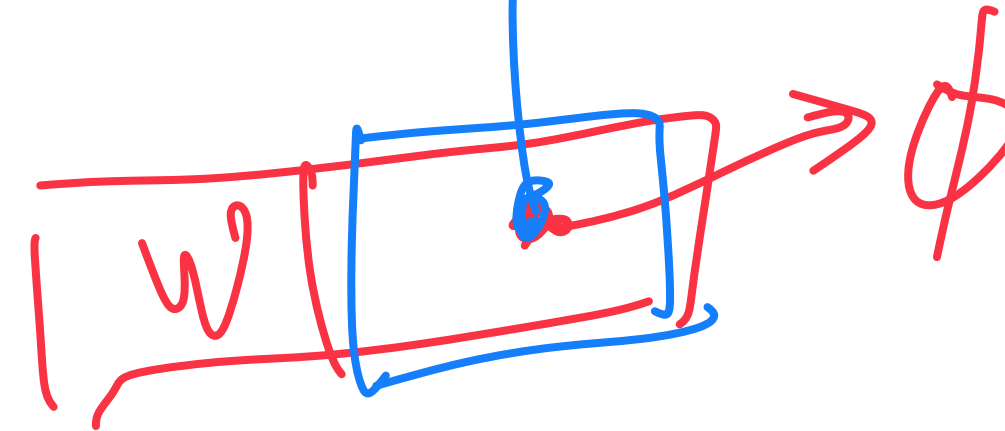
Sentinel Node



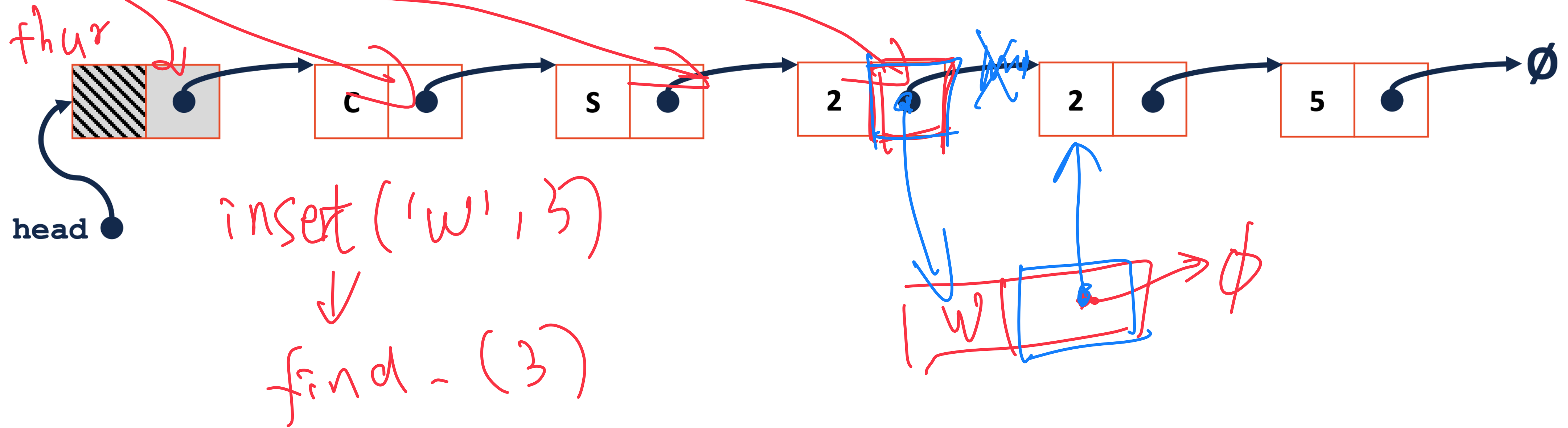
head

insert('w', 3)

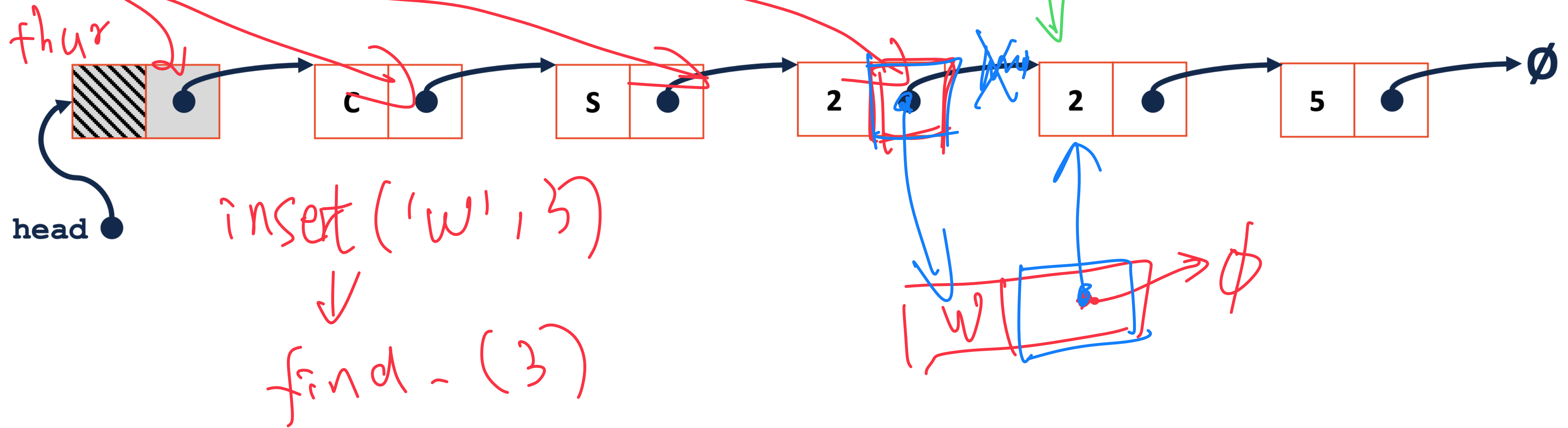
↓
find - (3)



Sentinel Node



Sentinel Node



ListNode * &ptr = find_index
return by reference
value

```

1  #include "List.h"
2
3  ListNode *& List::find_(unsigned index) const {
4      ListNode *thru = head;
5      for (unsigned i=0; i < index; i++) {
6          thru = thru->next;
7      }
8      return thru;
9  }

```

```

10
11 T& List::remove(unsigned index) {

```

remove: ① find ② delete

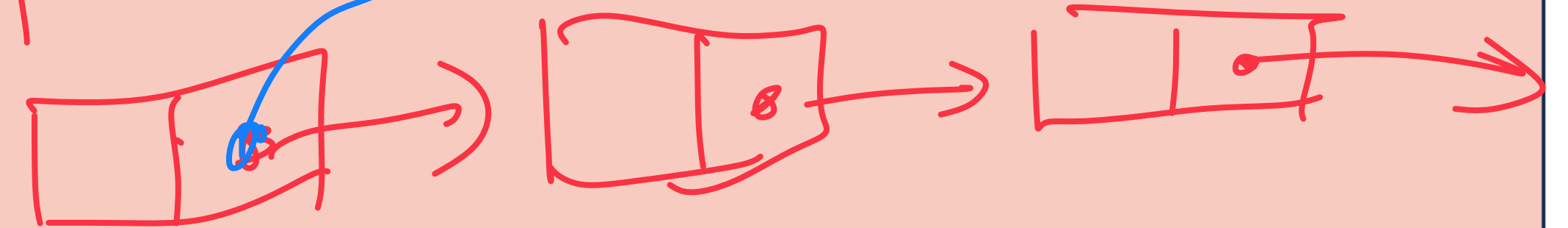
```

12     ListNode * & ptr = find_(index);
13
14
15
16
17
18 }

```

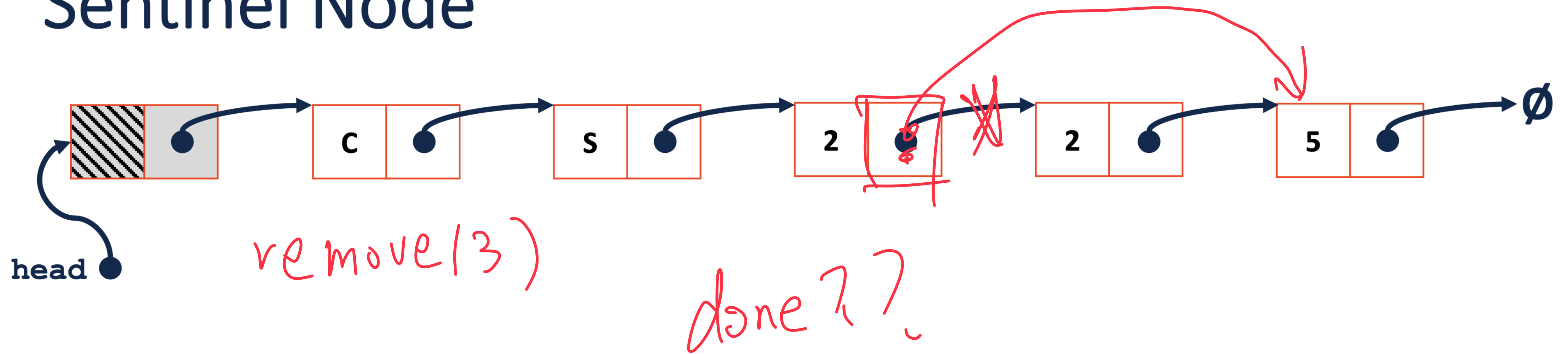
```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     ListNode *thru = head;
5     for (unsigned i=0; i < index; i++) {
6         thru = thru->next;
7     }
8     return thru;
9 }
```

remove: ① find ② delete

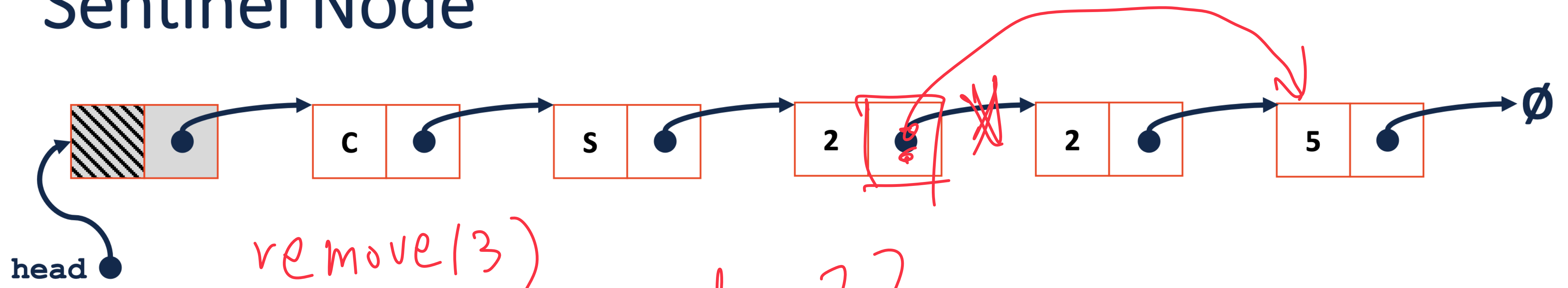


```
11 T& List::remove(unsigned index) {
12     ListNode * & ptr = find_(index);
13
14     ptr = ptr->next;
15
16
17
18 }
```


Sentinel Node

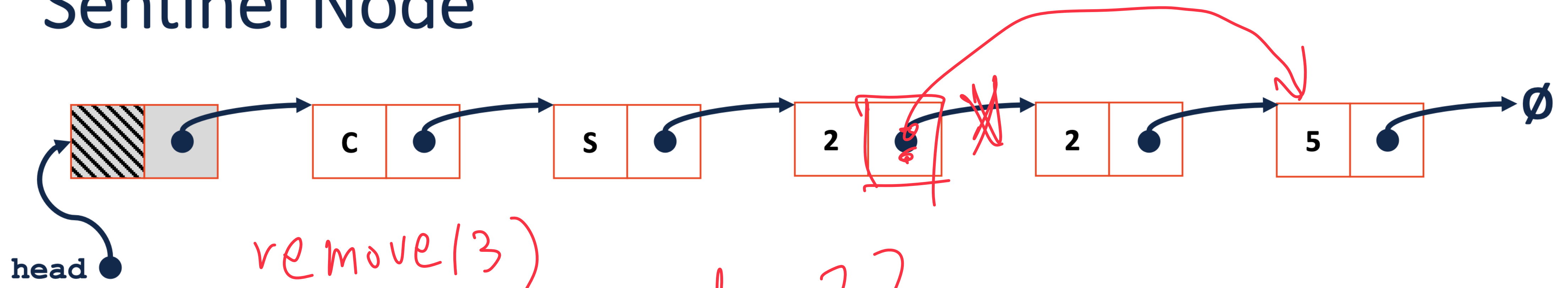


Sentinel Node



① memory leak

Sentinel Node



done??

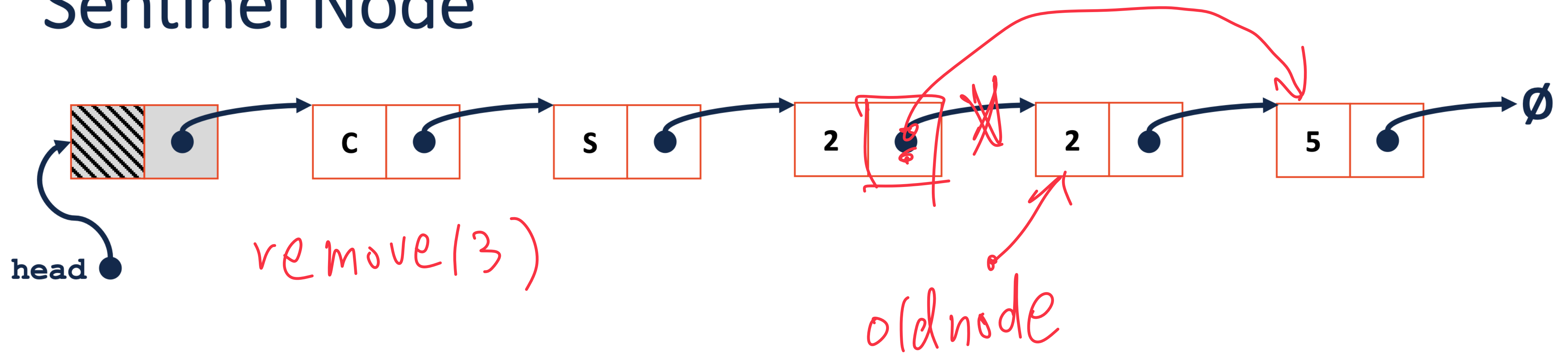
① memory leak

② return removed value

→ T by reference

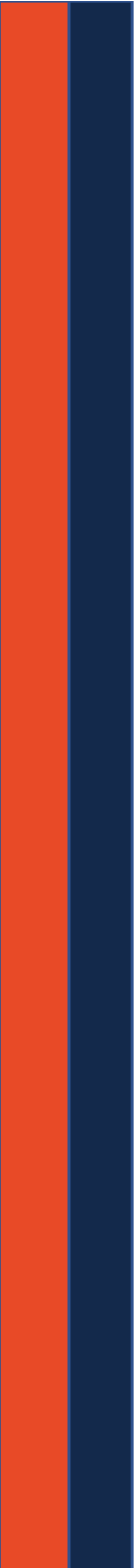

```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     /* */
5 }
6
7 T& List::remove(unsigned index) {
8     ListNode *ptr = find_(index)
9     ListNode *oldnode = ptr
10
11
12
13
14
15
16
17
18 }
```

Sentinel Node




```
1 #include "List.h"
2
3 ListNode *& List::find_(unsigned index) const {
4     /* */
5 }
6
7 T& List::remove(unsigned index) {
8     ListNode *ptr = find_(index);
9
10    ListNode *oldnode = ptr;
11
12    ptr = ptr->next;
13
14    T& data = oldnode->data;
15
16    delete oldnode;
17
18    return data;
```

return data; give this object back unchanged



List Implementations

1. Linked List

2. Array

```
1  #ifndef LIST_H
2  #define LIST_H
3
4  template <class T>
5  class List {
6      public:
7          /* ... */
8
28     private:
29
30         T * array;
31
32         unsigned count;
33
34         unsigned capacity;
35
36
37
38
39
40 };
41
42 #endif
```


Array Implementation

c	s	2	2	5
[0]	[1]	[2]	[3]	[4]

- $\text{get}(3) : \text{array}(3)$, $O(1)$

Array Implementation

c	s	2	2	5
[0]	[1]	[2]	[3]	[4]

- `get(3)` : `array(3)` , $O(1)$

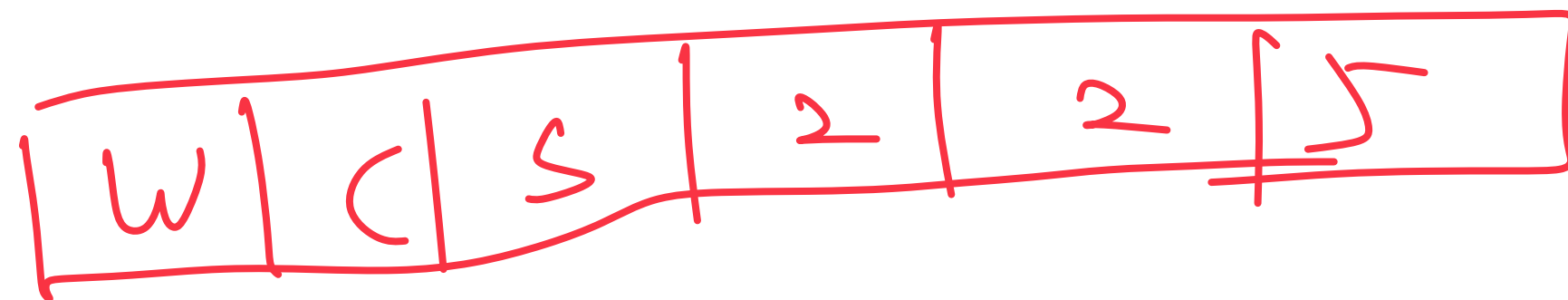
- `InsertAt front` : `insertAtfront('w')`

Array Implementation

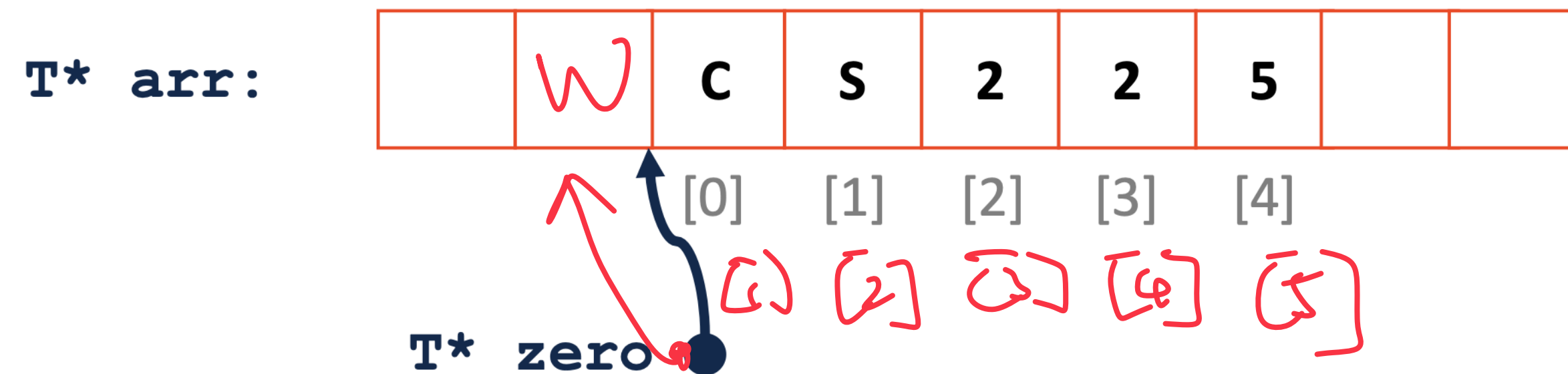
c	s	2	2	5
[0]	[1]	[2]	[3]	[4]

- $\text{get}(3) : \text{array}(3)$, $O(1)$

- Insert At front : $O(n)$



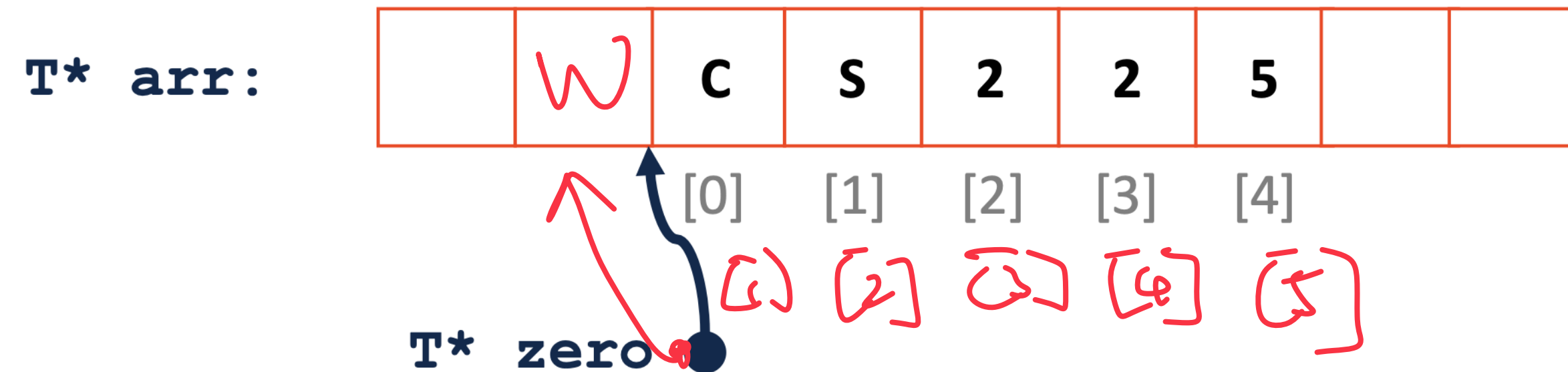
Array Implementation



$zero[1] == 'c'$

$insertAtFront \rightarrow O(1)^*$

Array Implementation



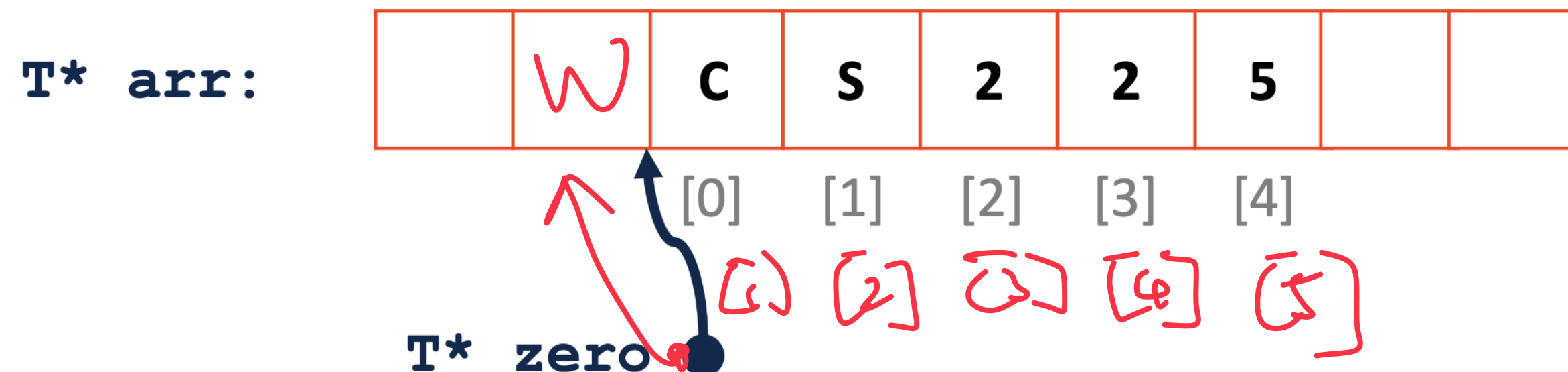
$zero[1] == 'c'$

$insertAtFront \rightarrow O(1)^*$

ran out of space

↓
resize array

Array Implementation



$zero[1] = 'C'$

insertAtFront $\rightarrow O(1)^*$

Resize: ① Double size of Array.
② Copy existing data to new array.

ran out of space
 $\downarrow$
resize array

Array Implementation

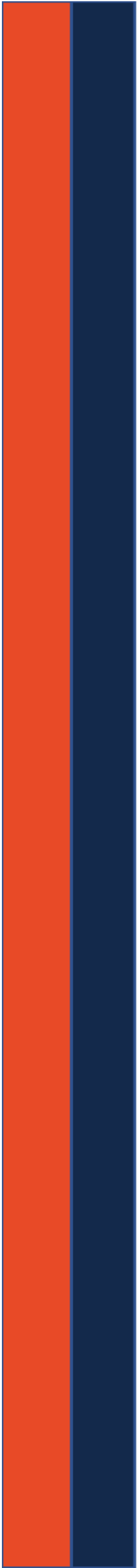
	Singly Linked List	Array
Insert/Remove at front		
Insert at given location		
Remove at given location		
Insert at arbitrary location		
Remove at arbitrary location		

Array Implementation

	Singly Linked List	Array
Insert/Remove at front		
Insert at given location		
Remove at given location		
Insert at arbitrary location		
Remove at arbitrary location		

Array Implementation

	Singly Linked List	Array
Insert/Remove at front	$O(1)$	$O(1)^*$, we may need to allocate extra space
Insert at given location	$O(1)$ if you can have a clever algorithm for knowing where to insert	$O(n)$ as in the worst case all elements have to be moved
Remove at given location	$O(1)$, again, given location	$O(n)$ as in the worst case all elements have to be moved
Insert at arbitrary location	$O(n)$, as we may need to visit every element to find the required location	$O(n)$ as in the worst case all elements have to be moved
Remove at arbitrary location	$O(n)$	$O(n)$ as in the worst case all elements have to be moved



CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Passing parameters by reference:

If a function receives a reference to a variable, it can modify the value of the variable.

```
#include <iostream>
using namespace std;

void swap(int& first, int& second)
{
    int temp = first;
    first = second;
    second = temp;
}

int main()
{
    int a = 2, b = 3;
    swap(a, b);
    cout << a << " " << b;
    return 0;
}
```

3 2

Passing parameters by reference:

If a function receives a reference to a variable, it can modify the value of the variable.

```
1  #include <iostream>
2  using namespace std;
3
4  void swap(int first, int & second)
5  {
6      int temp = first;
7      first = second;
8      second = temp;
9  }
10
11 int main()
12 {
13     int a = 2, b = 3;
14     swap(a, b);
15     cout << a << " " << b;
16     return 0;
17 }
18
```

2 2

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         void push (T& t);
8
9
10
11
12
13
14
15
16
17     private:
18
19
20 };
21
22 #endif
```


Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7
8
9
10
11
12
13
14
15
16
17     private:
18
19
20 };
21
22 #endif
```

void push (T& t);

T& pop (); — Return

the value on the top of stack

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7
8
9
10
11
12
13
14
15
16
17     private:
18
19
20 };
21
22 #endif
```

void push (T& t);

T& pop (); — Return

bool isEmpty (); const;

Stack (); — create an empty stack

the value on the top of stack

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7
8
9
10
11
12
13
14
15
16
17     private:
18
19
20 };
21
22 #endif
```

void push (T& t);

T& pop (); — Return

bool isEmpty (); const;

Stack (); — create an empty stack

more operation: clear, pick

— based on these operations.

Stack.h

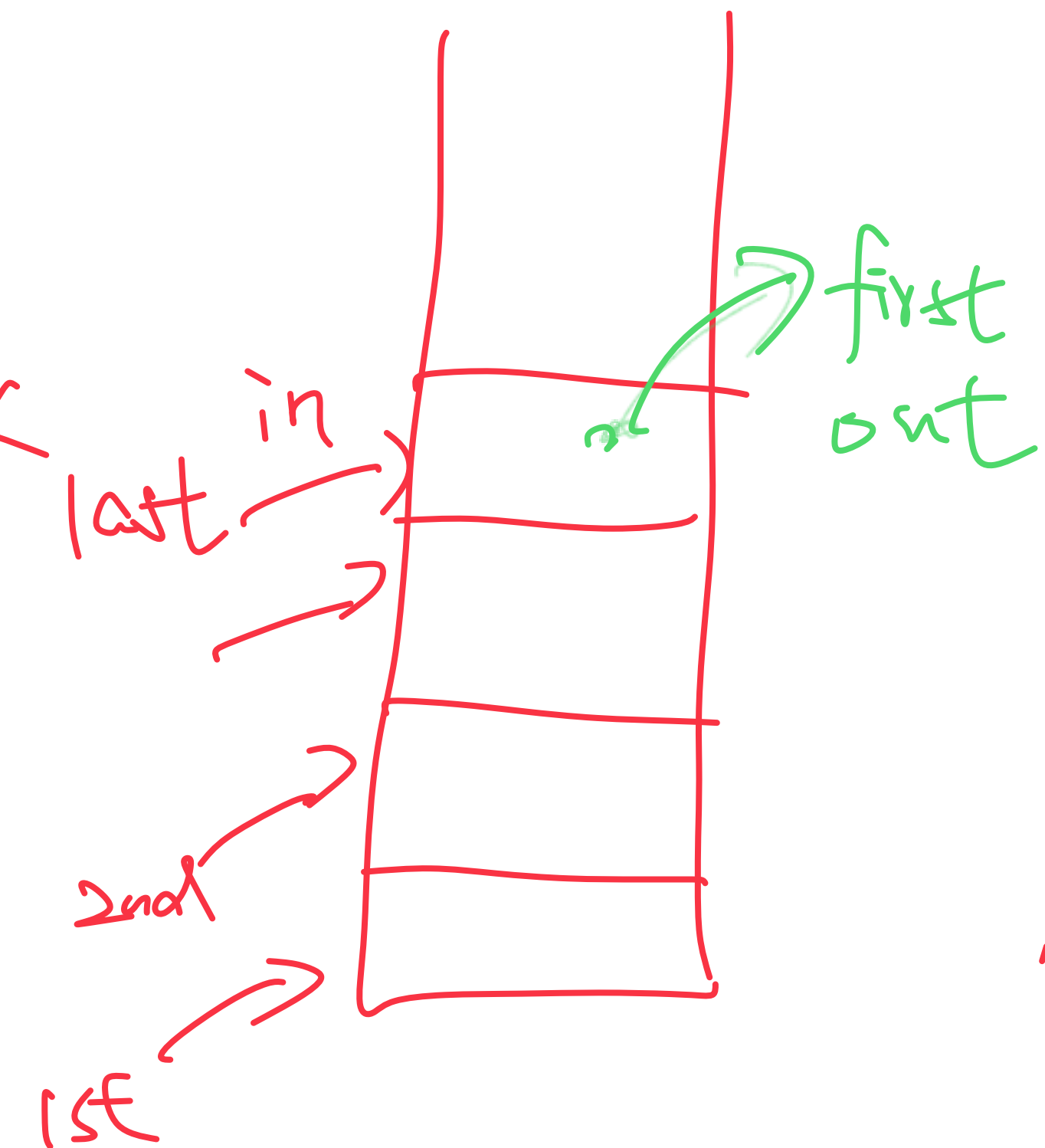
```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         void push (T& t);
8
9         T& pop ();
10
11         bool isEmpty () const;
12
13         Stack (); // create an empty stack
14
15     private:
16         ...
17
18
19
20 };
21
22 #endif
```

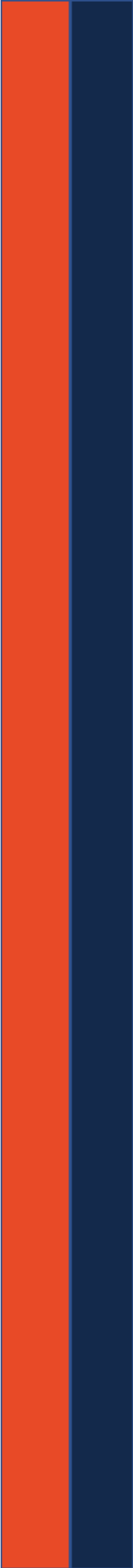
L	I	F	O
a	n	i	y
s		t	
t			

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         void push (T& t);
8
9         T& pop ();
10
11         bool is Empty ( ) const ;
12
13         Stack ( );    - create an empty stack
14
15     private:
16         ... -
17
18
19
20 };
21
22 #endif
```

L	I	F	O
a	n	i	y
s		r	t
t		s	
		t	



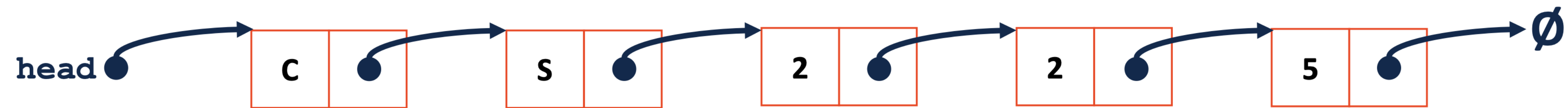


Stack Implementations

1. *Linked List*

2. *Array*

Linked Memory

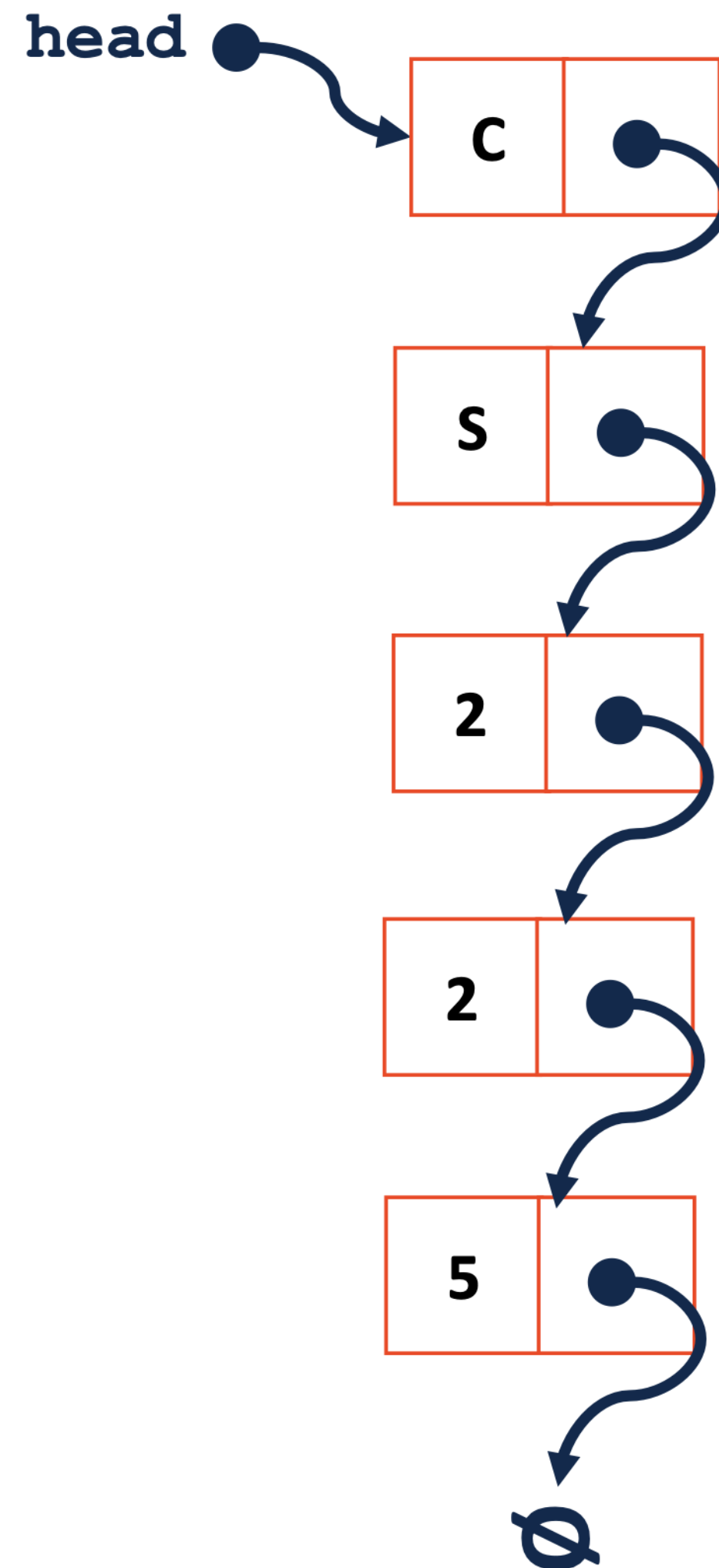


rotate it

horizontally → vertically

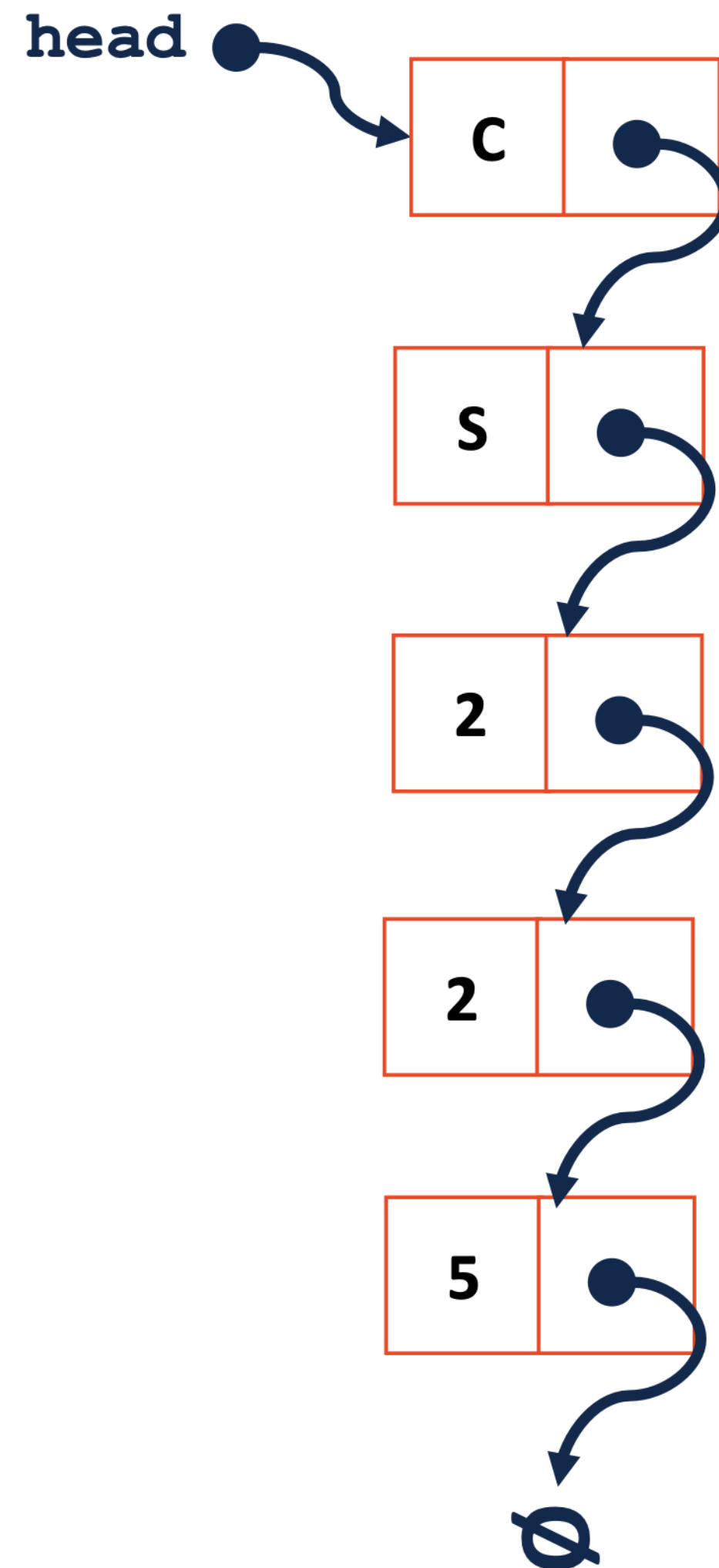
Stack - Linked List

struct StackNode {



}

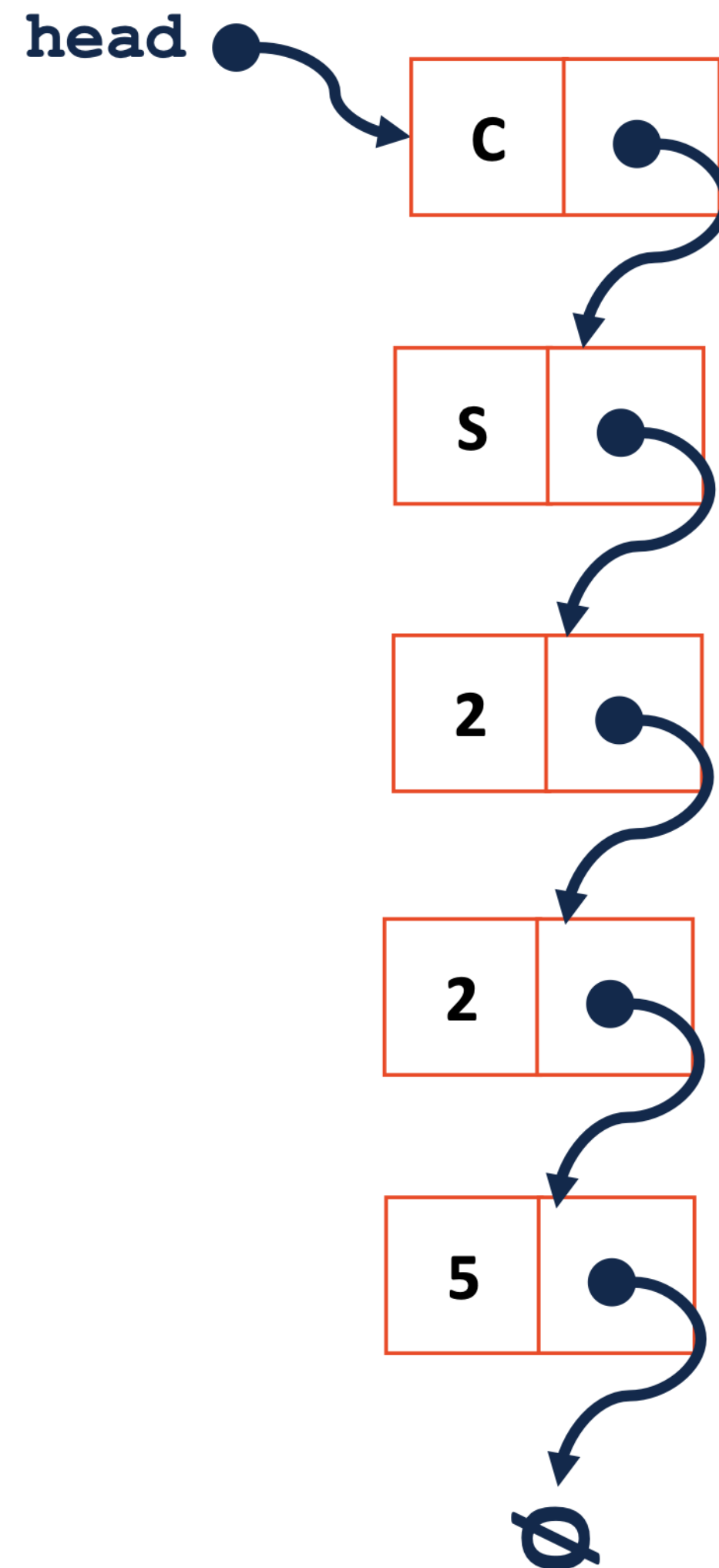
Stack - Linked List



```
struct StackNode {  
    T& t;  
    StackNode* next;  
};
```

}

Stack - Linked List



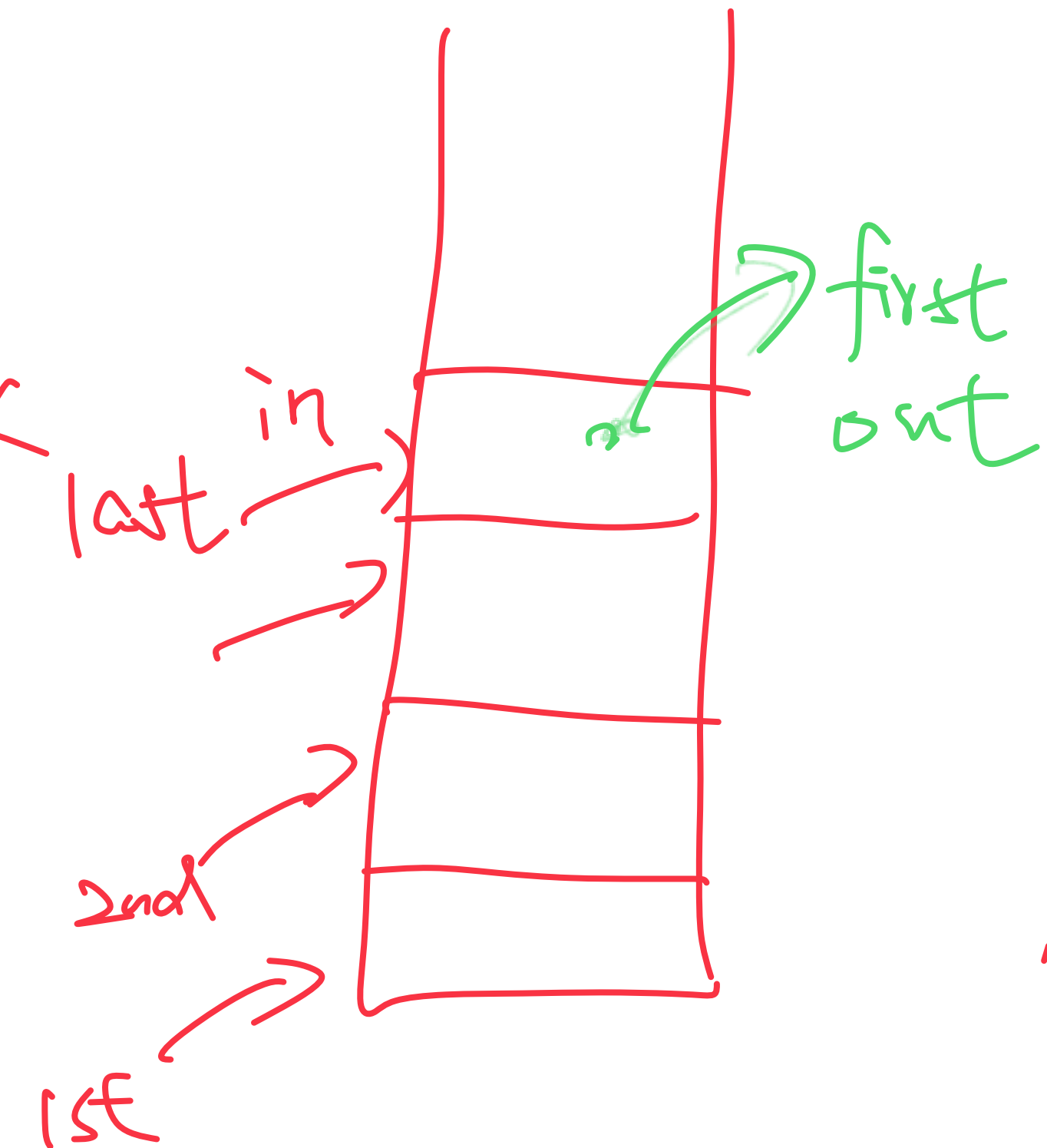
```
struct StackNode {  
    T& t;  
    StackNode* next;  
};  
StackNode(T& t): .....
```

}

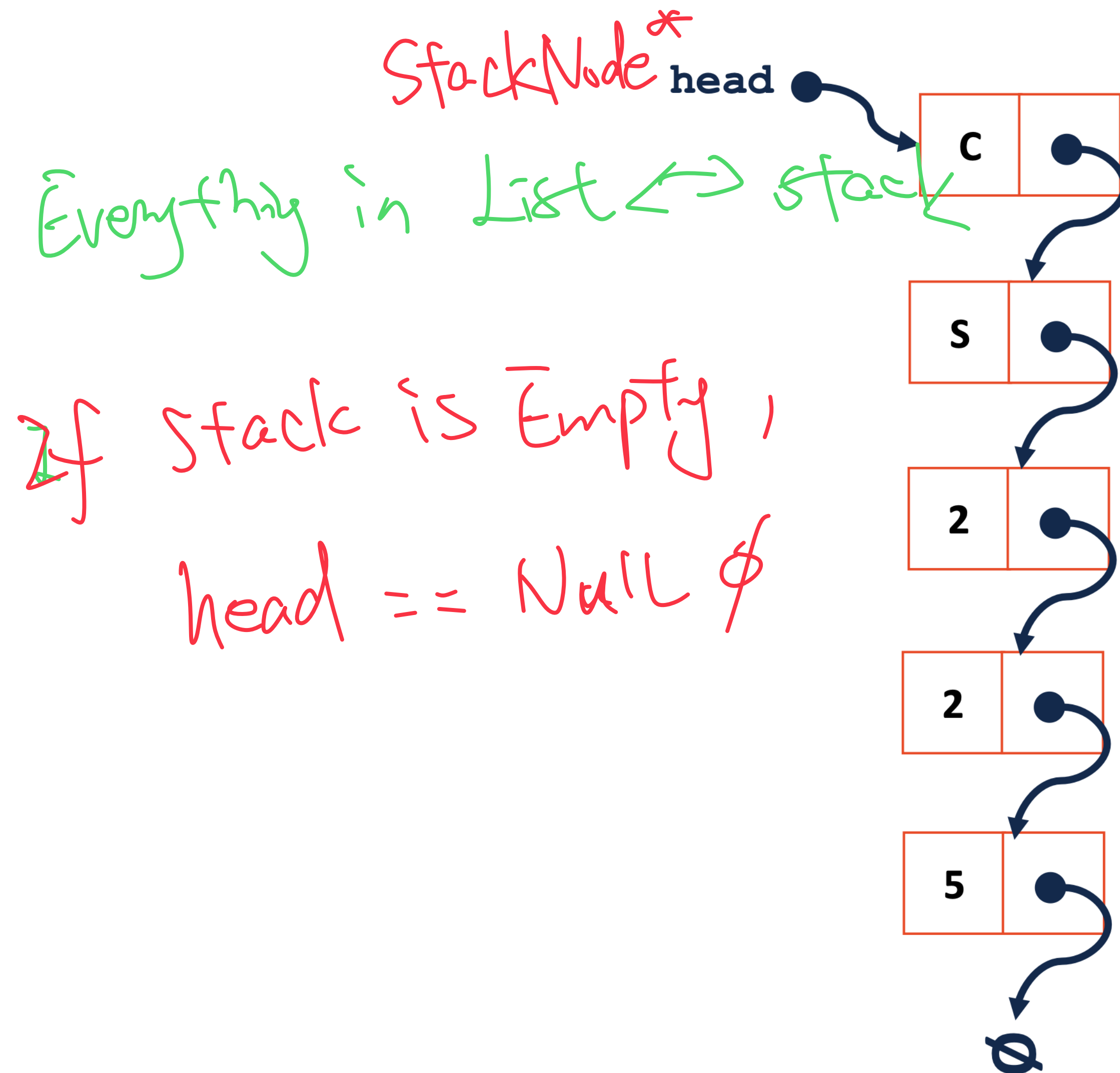
Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         void push (T& t);
8
9         T& pop ();
10
11         bool isEmpty () const;
12
13         Stack (); // create an empty stack
14
15     private:
16         struct
17
18         struct
19
20 };
21
22 #endif
```

L	I	F	O
a	n	i	y
s		r	t
t		s	
		t	



Stack - Linked List



```
struct StackNode {  
    T& t;  
    StackNode * next;  
};  
StackNode(T& t): .....
```

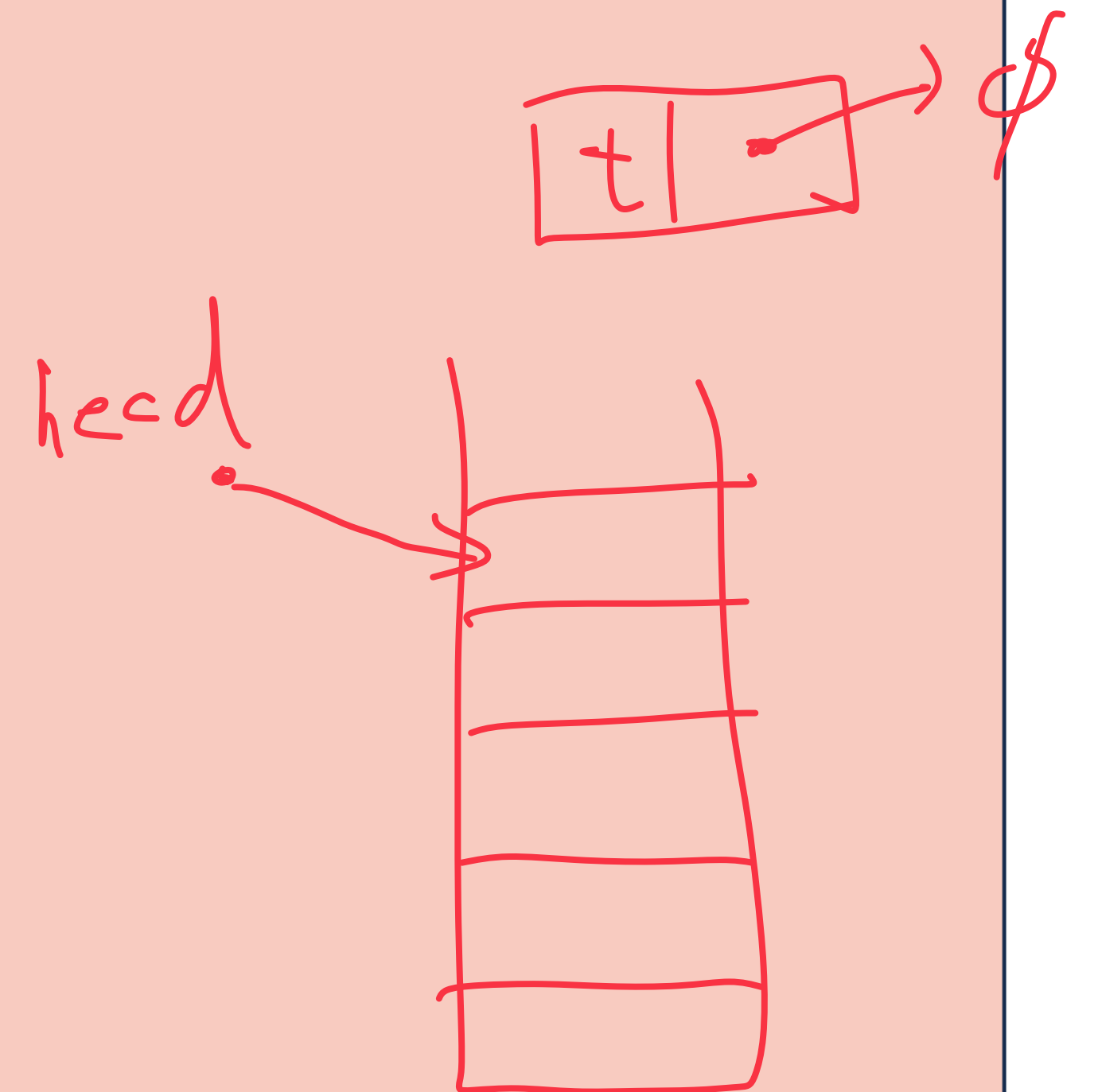
}

```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4     Stack<Node> *Node = new Stack Node(t);
5     node -> next = head;
6     head = node ;
7 }
8
9
10
11 T & Stack<T>::pop() {
12
13
14
15
16
17
18
19 }
```

```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     StackNode *Node = new StackNode(t);
5     Node->next = head;
6     head = Node ;
7 }
8
9
10
11 T & Stack<T>::pop() {
12
13
14
15
16
17
18
19 }
```

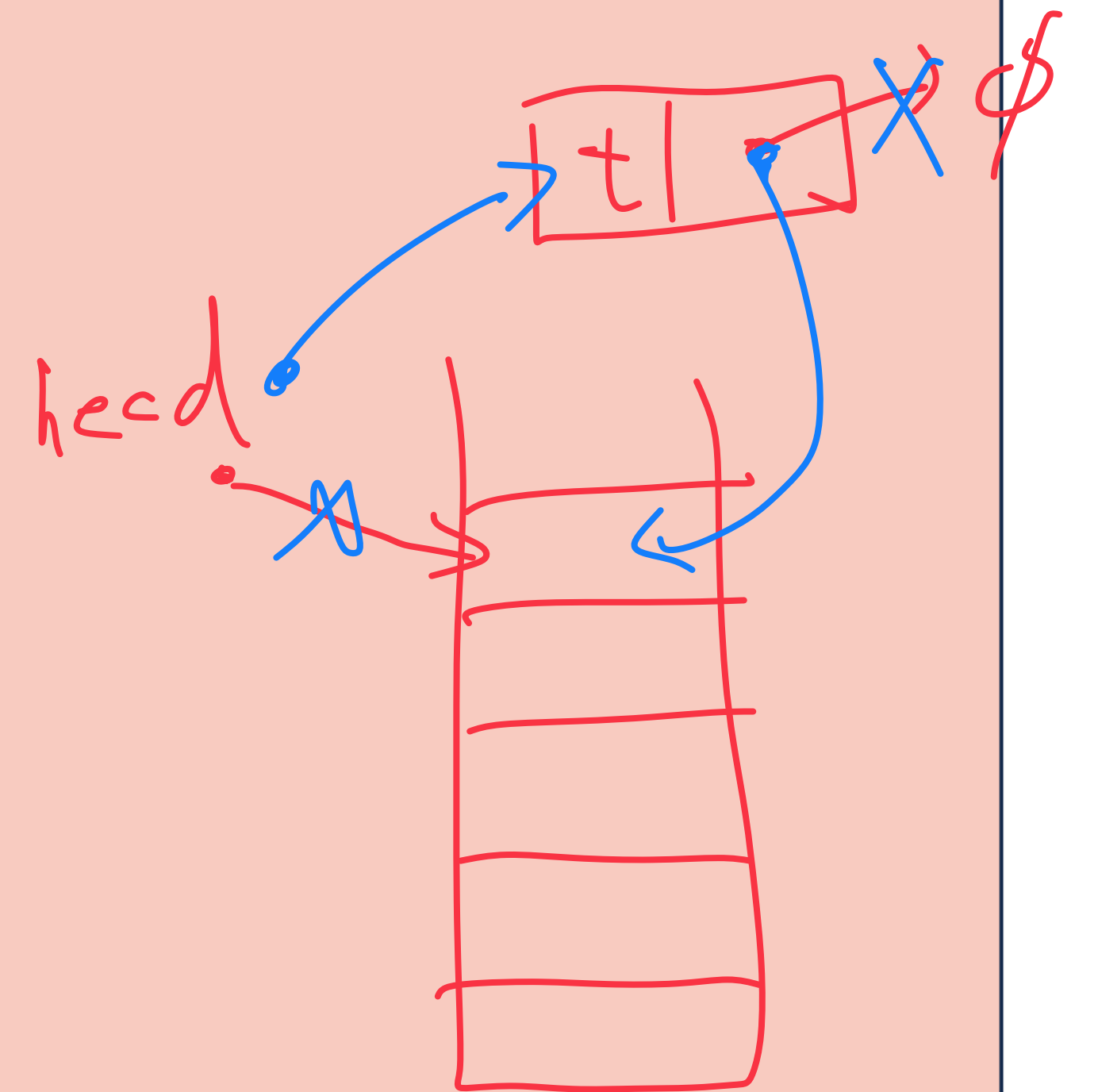

Stack.cpp

```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     StackNode *Node = new StackNode(t);
5     Node->next = head;
6     head = Node;
7 }
8
9
10
11 T & Stack<T>::pop() {
12
13
14
15
16
17
18
19 }
```

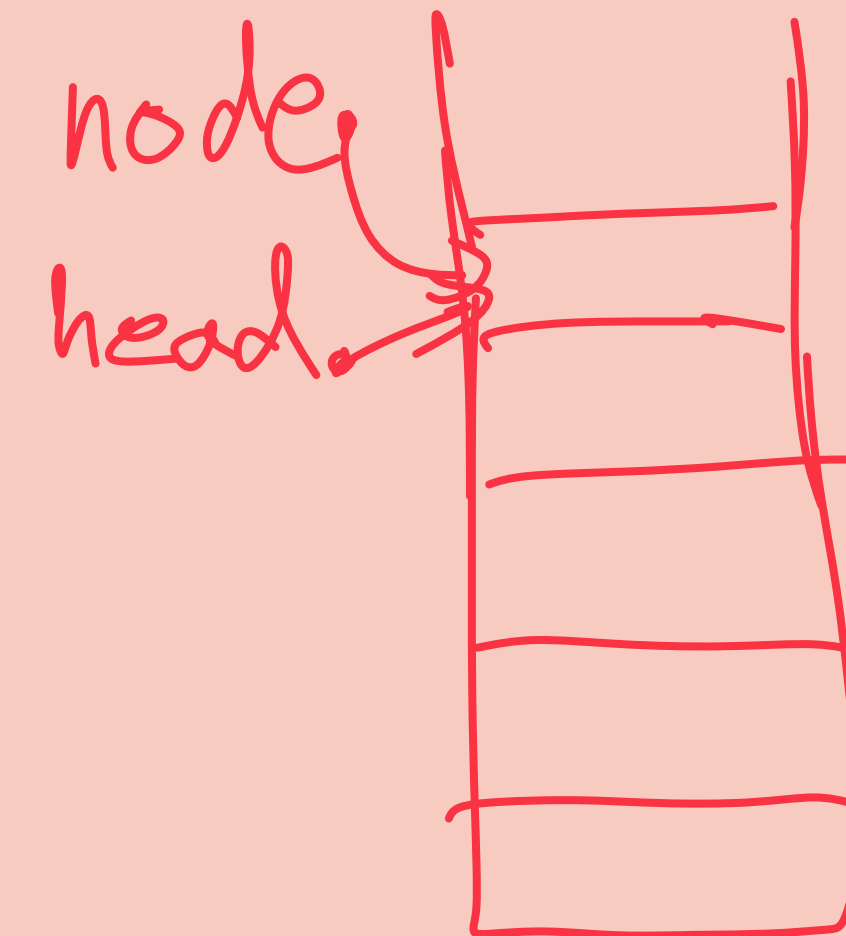


Stack.cpp

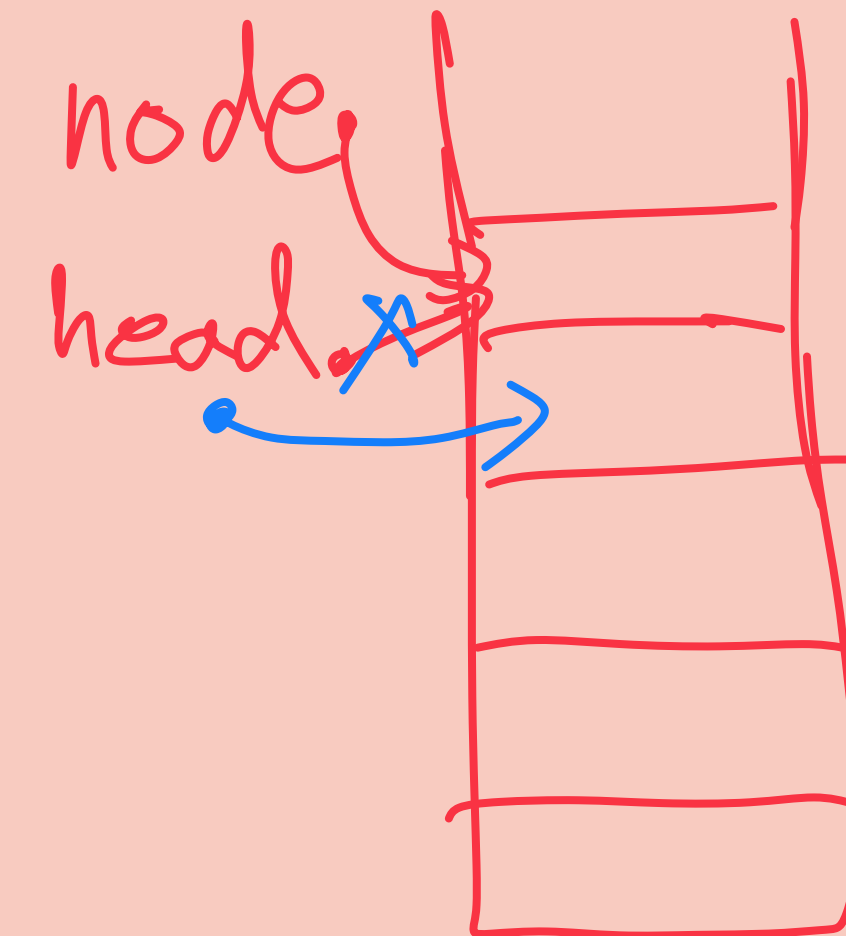
```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     stack<Node *Node> = new Stack Node(t);
5     node → next = head;
6     head = node ;
7 }
8
9
10
11 T & Stack<T>::pop() {
12
13
14
15
16
17
18
19 }
```



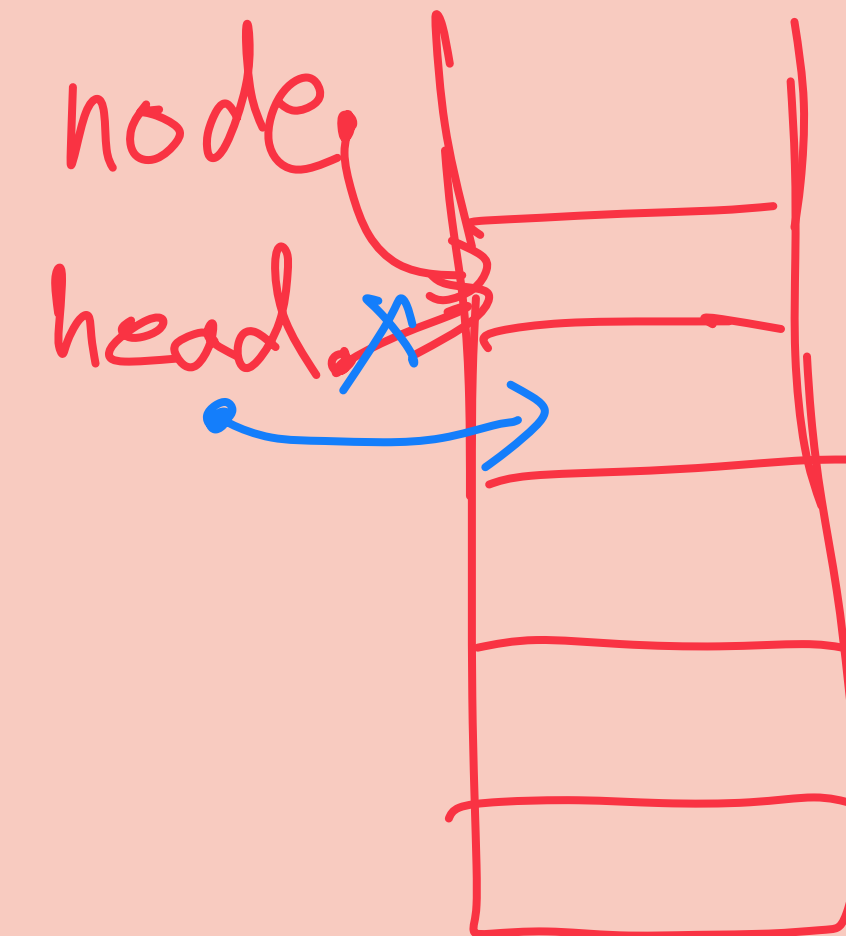

```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     StackNode *Node = new StackNode(t);
5     Node->next = head;
6     head = Node;
7 }
8
9
10
11 T & Stack<T>::pop() {
12     StackNode *Node = head;
13
14
15
16
17
18
19 }
```



```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     StackNode *Node = new StackNode(t);
5     Node->next = head;
6     head = Node;
7 }
8
9
10
11 T & Stack<T>::pop() {
12     StackNode *Node = head;
13     head = head->next;
14 }
15
16
17
18
19 }
```



```
1 #include "Stack.h"
2 template <class T>
3 void Stack<T>::push(T & t) {
4     StackNode *Node = new StackNode(t);
5     node->next = head;
6     head = node ;
7 }
8
9
10
11 T & Stack<T>::pop() {
12     structNode *Node = head;
13     head = head->next;
14     T& data = node->data;
15     delete node;
16     return data;
17 }
18
19
```



Array Implementation

T* arr:

c	s	2	2	5		
[0]	[1]	[2]	[3]	[4]		



count == 5

Next element to insert:

arr []

Array Implementation

T* arr:

c	s	2	2	5		
[0]	[1]	[2]	[3]	[4]		

count == 5

Next element to insert:

arr[count];

pop(), remove:

arr[];

Array Implementation

T* arr:

c	s	2	2	5		
[0]	[1]	[2]	[3]	[4]		

count == 5

Next element to insert:

arr[count-1];

pop(), remove:

arr[count-1];

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         Stack();
8         Stack(const Stack &other);
9         ~Stack();
10        Stack& operator=(const Stack &other);
11
12        void push(T & t);
13        T & pop();
14        bool isEmpty() const;
15
16    private:
17
18
19
20 };
21
22 #endif
```

rule of three

Stack ADT

implement with array

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         Stack();
8         Stack(const Stack &other);
9         ~Stack();
10        Stack& operator=(const Stack &other);
11
12        void push(T & t);
13        T & pop();
14        bool isEmpty() const;
15
16    private:
17        unsigned size_;
18        unsigned count_;
19        T* array_;
20 };
21
22 #endif
```

rule of three

Stack ADT

implement with array

Stack.cpp

```
1  #include "Stack.h"
2
3  void Stack<T>::push(T & t) {
4      arr[count] = t;
5  }
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15
16
17
18
19 }
```

```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4
5     arr[count++] = t;
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15
16
17
18
19 }
```

① yield current value

② add one

post increment

```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4     arr[count++] = t;    post increment
5
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15     return arr[count];
16
17
18
19 }
```

① yield current value
② add one


```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4     arr[count++] = t;
5
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15
16
17
18
19 }
```

① yield current value

② add one

post increment

pre-decrement

① subtract one

② yield value

```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4
5
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15
16
17
18
19 }
```

arr[count++] = t;

if (count + 1 == size) {

② add one

post increment

① yield current value

return arr[--count];

pre-decrement

① subtract one

② yield value

```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4
5
6
7
8
9
10 }
11
12 T & Stack<T>::pop() {
13
14
15
16
17
18
19 }
```

arr[count++] = t;

if (count + 1 == size) {

T* newArr = new T[size * 2];
size = size * 2;

return arr[--count];

① yield current value

② add one

post increment

pre-decrement

① subtract one

② yield value


```
1 #include "Stack.h"
2
3 void Stack<T>::push(T & t) {
4     arr[count++] = t;
5     if (count + 1 == size) {
6         T* newArr = new T[size * 2];
7         size = size * 2;
8         copy(arr, newArr);
9     }
10 }
11
12 T & Stack<T>::pop() {
13     delete [] arr;
14     arr = newArr;
15     return arr[--count];
16 }
17
18
19 }
```

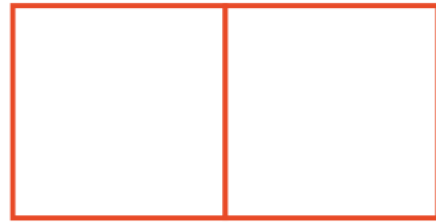
① yield current value
② add one

post increment

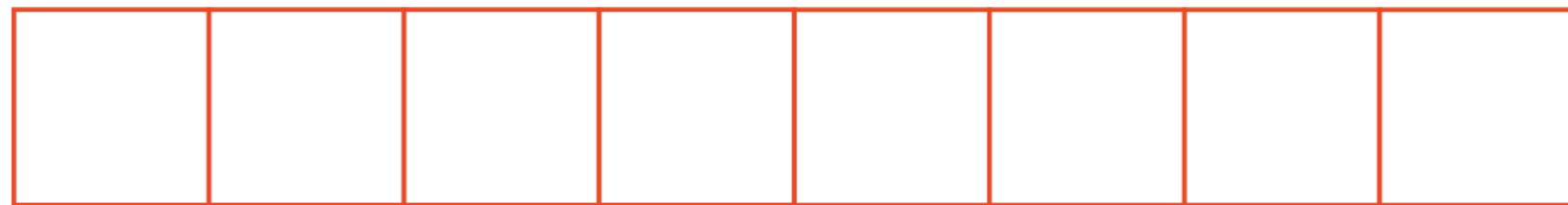
pre-decrement

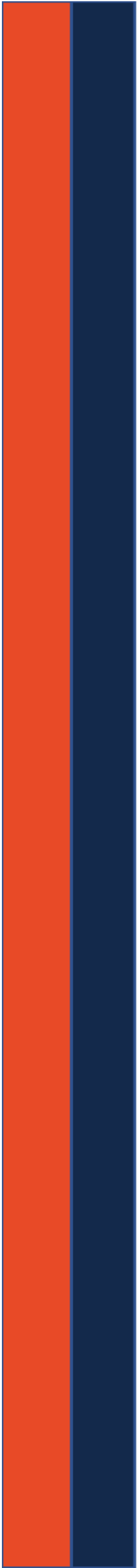
① subtract one
② yield value

Resize Strategy – Details



Resize Strategy – Details





CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         Stack();
8         Stack(const Stack &other);
9         ~Stack();
10        Stack& operator=(const Stack &other);
11
12        void push(T & t);
13        T & pop();
14        bool isEmpty() const;
15
16    private:
17        T * arr_;
18        unsigned size_, count_;
19
20 };
21
22 #endif
```

Any problems with
this implementation?


```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T * newArray = new T[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T * newArray = new T[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T * newArray = new T[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

delete [] arr;

RHS: object reference
LHS:

Stack.cpp

```
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         T * newArray = new T[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10        delete arr_;
11        arr_ = newArray;
12    }
13
14    // Insert (push) the element into the array-backed stack:
15    arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

RHS: object reference
LHS: array of objects $T^* \text{arr_}$

What would it do??
- object by reference being assigned to an object??

Example:

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int x = 10;
7      // ref is a reference to x.
8      int& ref = x;
9
10     // Value of x is now changed to 20
11     ref = 20;
12     cout << "x = " << x << '\n';
13
14     int y = ref;
15     cout << "y = " << y << '\n';
16     cout << "the address of x is " << &x << '\n';
17     cout << "the address of y is " << &y << '\n';
18     cout << "the address of ref is " << &ref << '\n';
19     return 0;
20 }
21
```

```
x = 20
y = 20
the address of x is 0x7fff17c593a8
the address of y is 0x7fff17c593ac
the address of ref is 0x7fff17c593a8
```


Stack.cpp

```
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         T * newArray = new T[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10        delete arr_;
11        arr_ = newArray;
12    }
13
14    // Insert (push) the element into the array-backed stack:
15    arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

delete [] arr;

RHS: object reference
LHS: array of objects.

Copy objects into arr_ instead of
storing the existing objects.

How to fix it ??
Not make a copy ??

Stack.h

```
1 #ifndef STACK_H
2 #define STACK_H
3
4 template <class T>
5 class Stack {
6     public:
7         Stack();
8         Stack(const Stack &other);
9         ~Stack();
10        Stack& operator=(const Stack &other);
11
12        void push(T & t);
13        T & pop();
14        bool isEmpty() const;
15
16    private:
17        T ** arr_;
18        unsigned size_, count_;
19
20 };
21
22 #endif
```

- make a double pointer

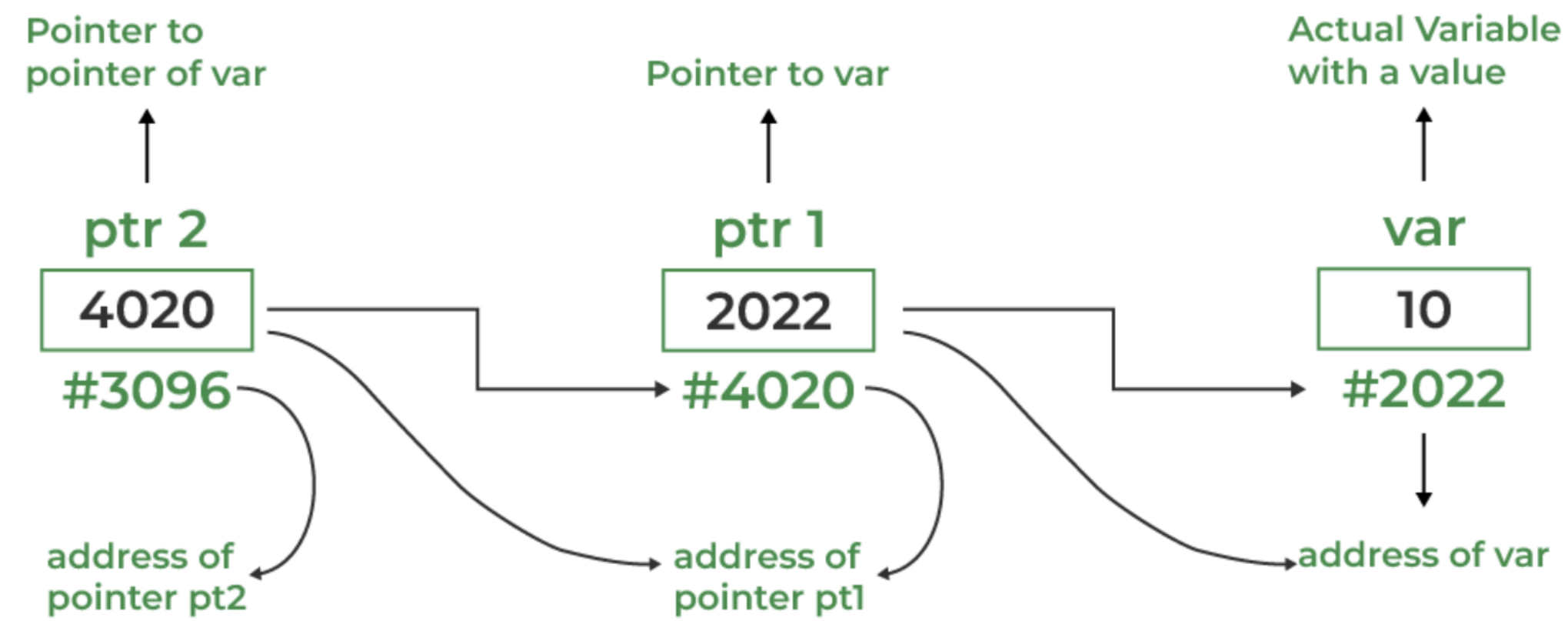
→ array of generic object pointers
but not objects.

Double Pointer:

The first pointer is used to store the address of the variables, and the second pointer stores the address of the first pointer.



Double Pointer



Stack.cpp

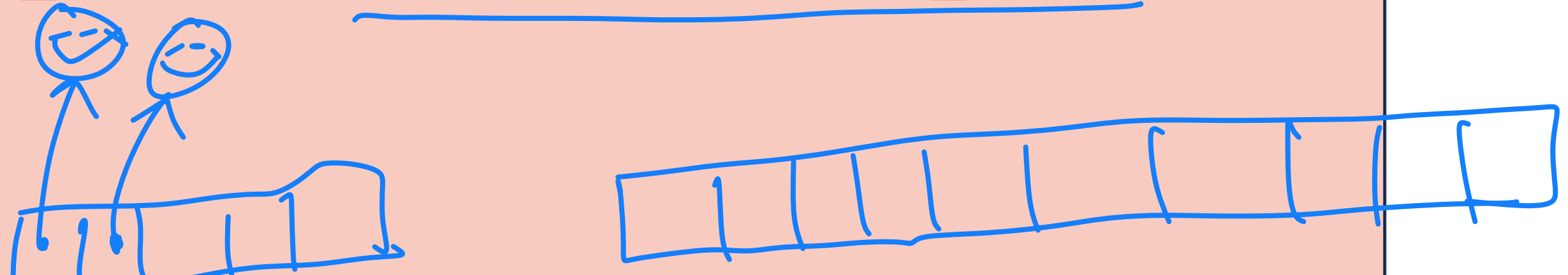
```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T **newArray = new T[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

```
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         T **newArray = new T[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10        delete arr_;
11        arr_ = newArray;
12    }
13
14    // Insert (push) the element into the array-backed stack:
15    arr[ count_++ ] = t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return arr[ --count_ ];
21 }
```

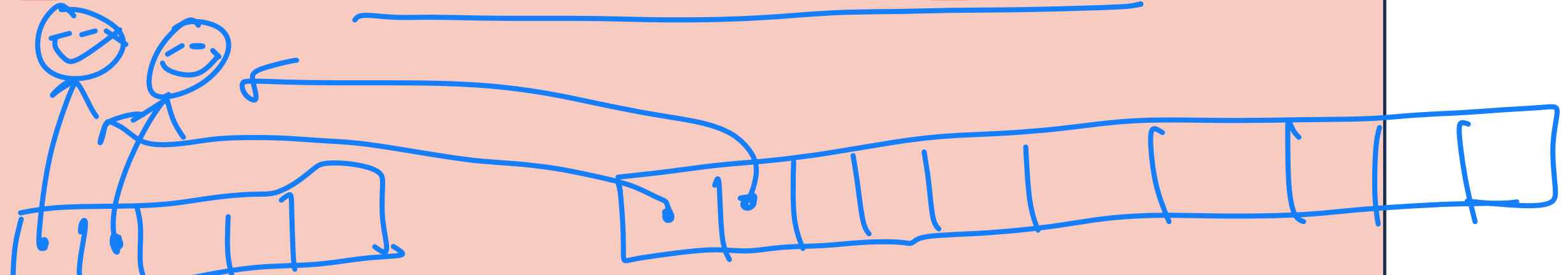
→ pointer of t


```
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         T **newArray = new T[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10        delete arr_;
11        arr_ = newArray;
12    }
13
14    // Insert (push) the element into the array-backed stack:
15    arr[ count_++ ] = &t; → pointer of t
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return *arr[ --count_ ]; → indirection — return objects.
21 }
```

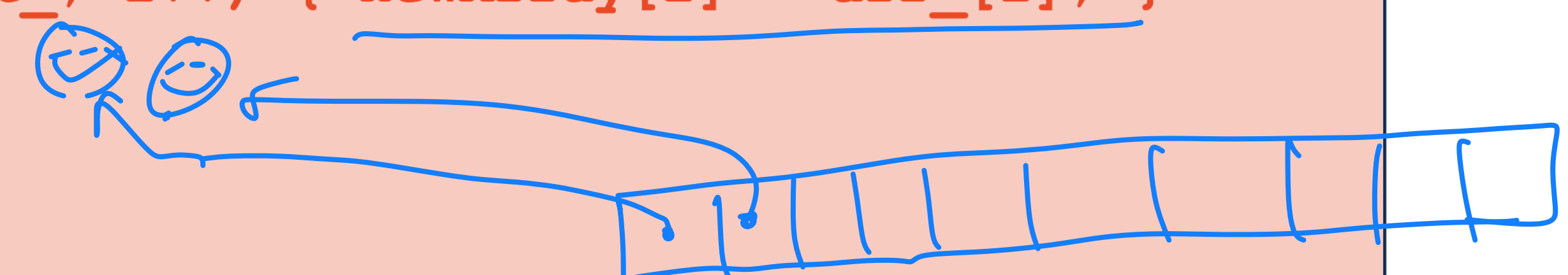
```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T ** newArray = new T*[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete [] arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = &t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return *(arr[ --count_ ]);
21 }
```



```
1  #include "Stack.h"
2
3  template <class T>
4  void Stack::push(T & t) {
5      // If we are about to overflow, double the size of the array:
6      if (count_ + 1 == size_) {
7          size_ *= 2;
8          T ** newArray = new T*[size_];
9          for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10         delete [] arr_;
11         arr_ = newArray;
12     }
13
14     // Insert (push) the element into the array-backed stack:
15     arr[ count_++ ] = &t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20     return *(arr[ --count_ ]);
21 }
```




```
1 #include "Stack.h"
2
3 template <class T>
4 void Stack::push(T & t) {
5     // If we are about to overflow, double the size of the array:
6     if (count_ + 1 == size_) {
7         size_ *= 2;
8         => T ** newArray = new T*[size_];
9         for (unsigned i=0; i < count_; i++) { newArray[i] = arr_[i]; }
10        => delete [] arr_;
11        arr_ = newArray;
12    }
13
14    // Insert (push) the element into the array-backed stack:
15    => arr[ count_++ ] = &t;
16 }
17
18 template <class T>
19 T & Stack::pop() {
20    => return *(arr[ --count_ ]);
21 }
```

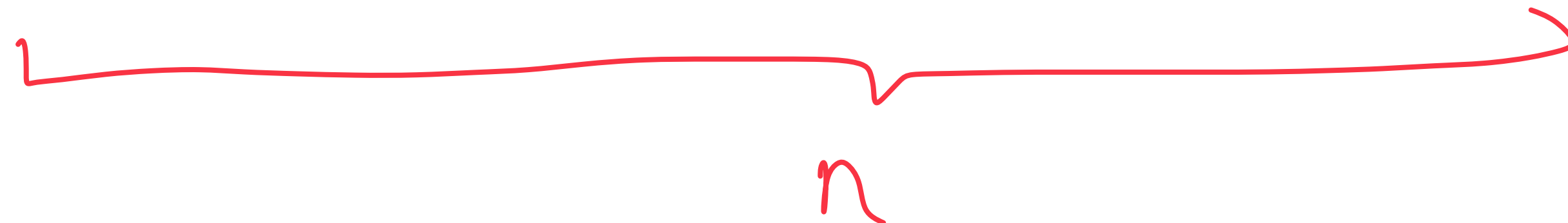


Resize Strategy – when full, add 2

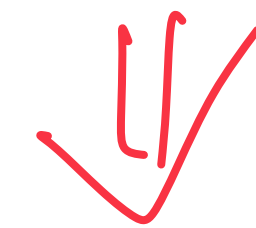


Resize Strategy – when full, add 2

k-levels



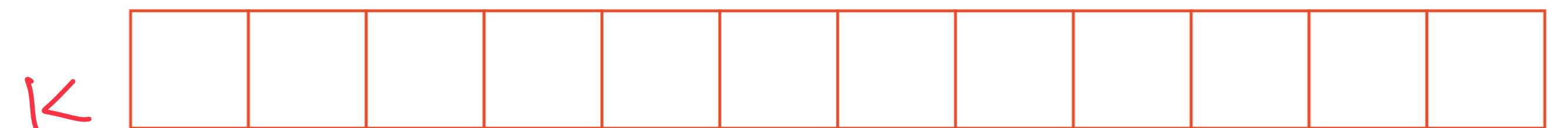
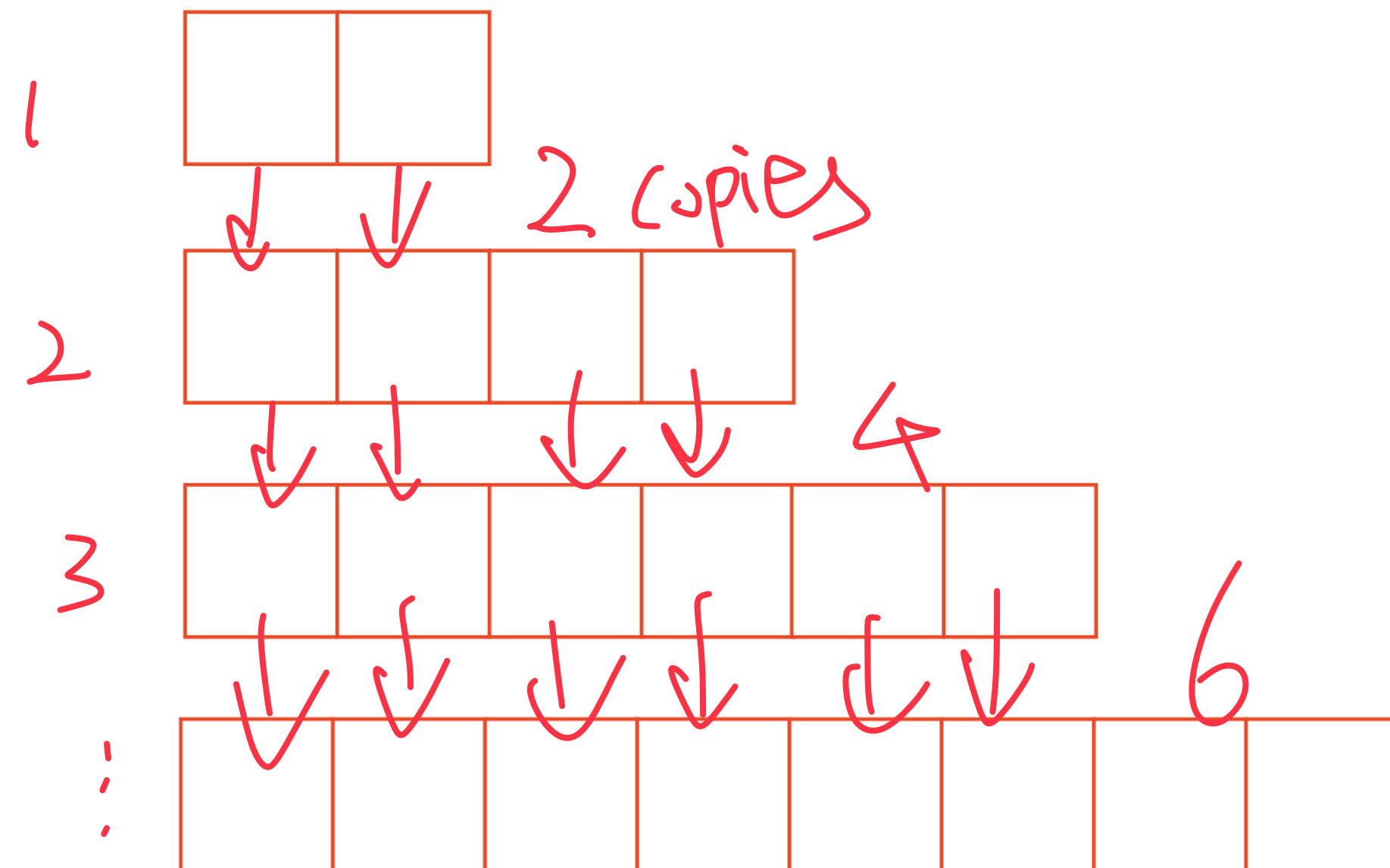
How much time will it
take to create elements
of size n ??



How many copies
I need to make??

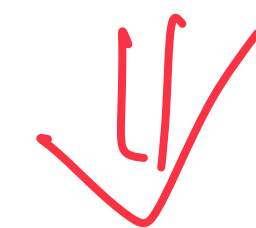
Resize Strategy – when full, add 2

k-levels



n

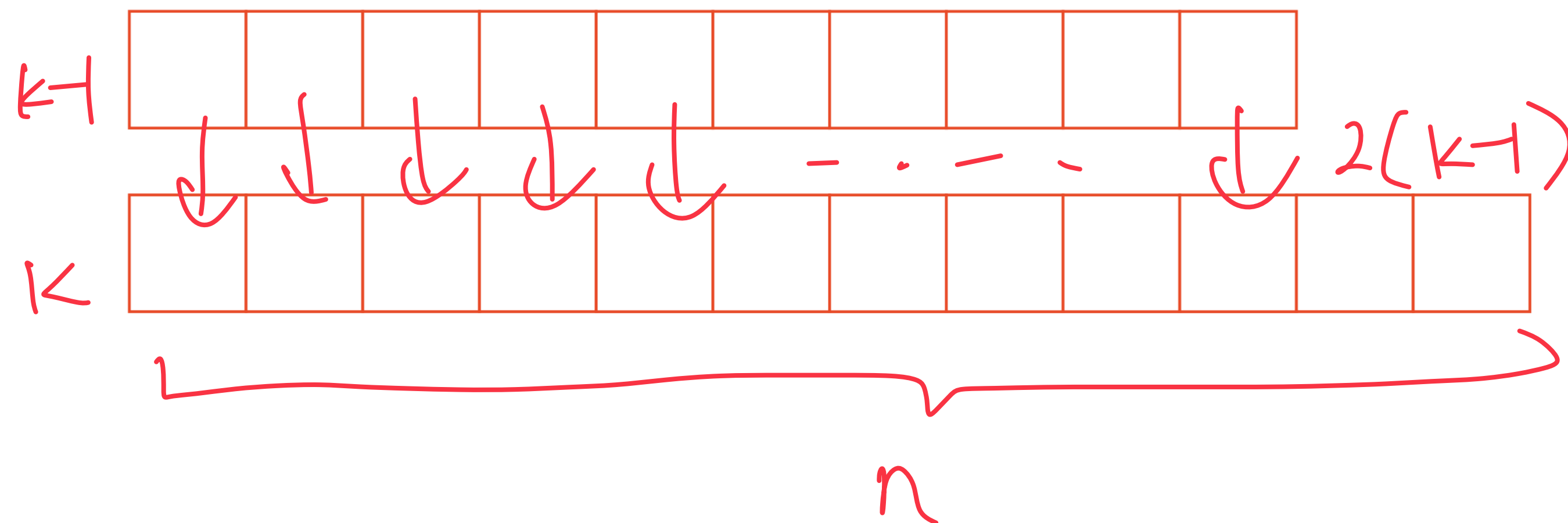
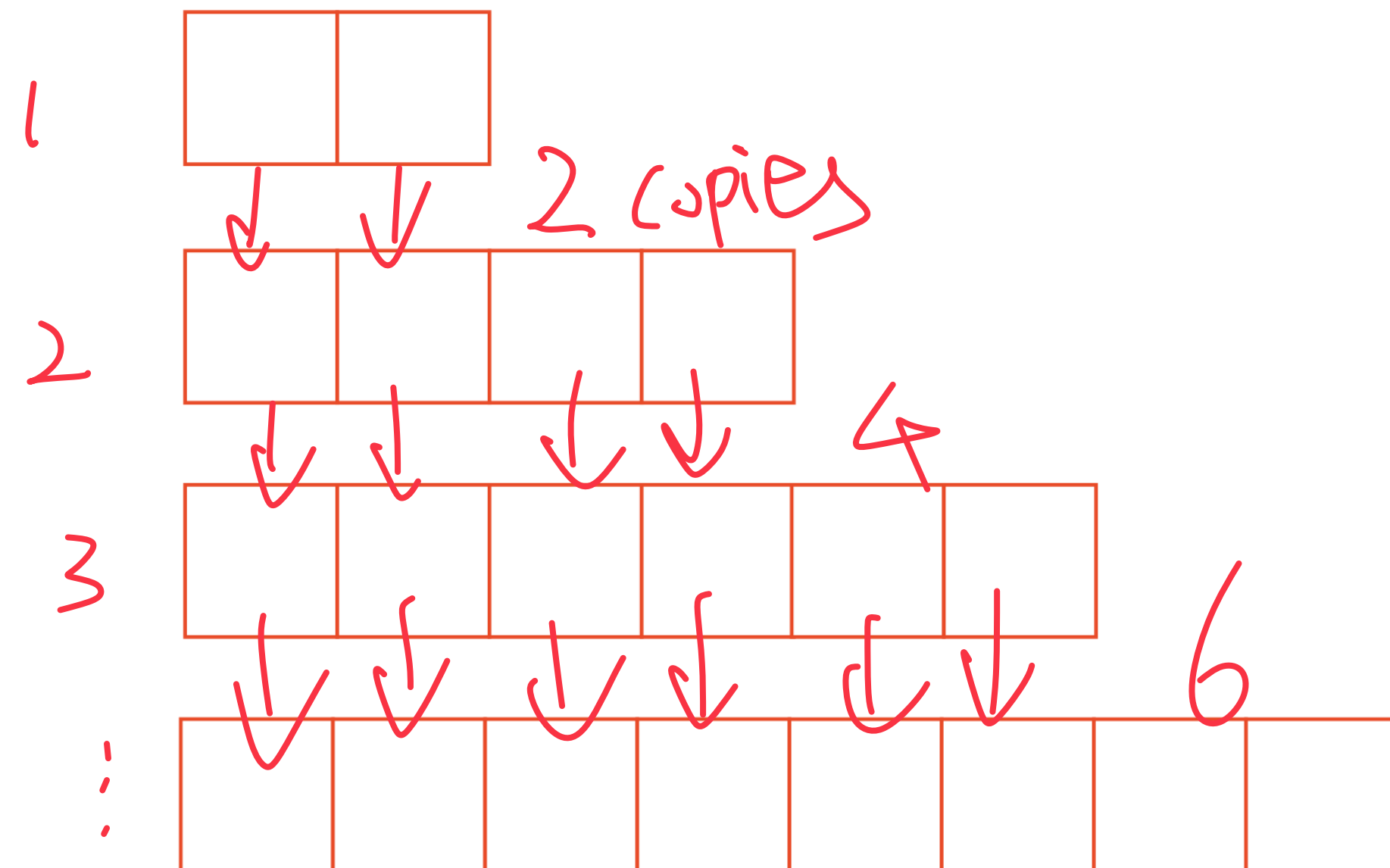
How much time will it
take to create elements
of size n ??



How many copies
I need to make??

Resize Strategy – when full, add 2

k-levels

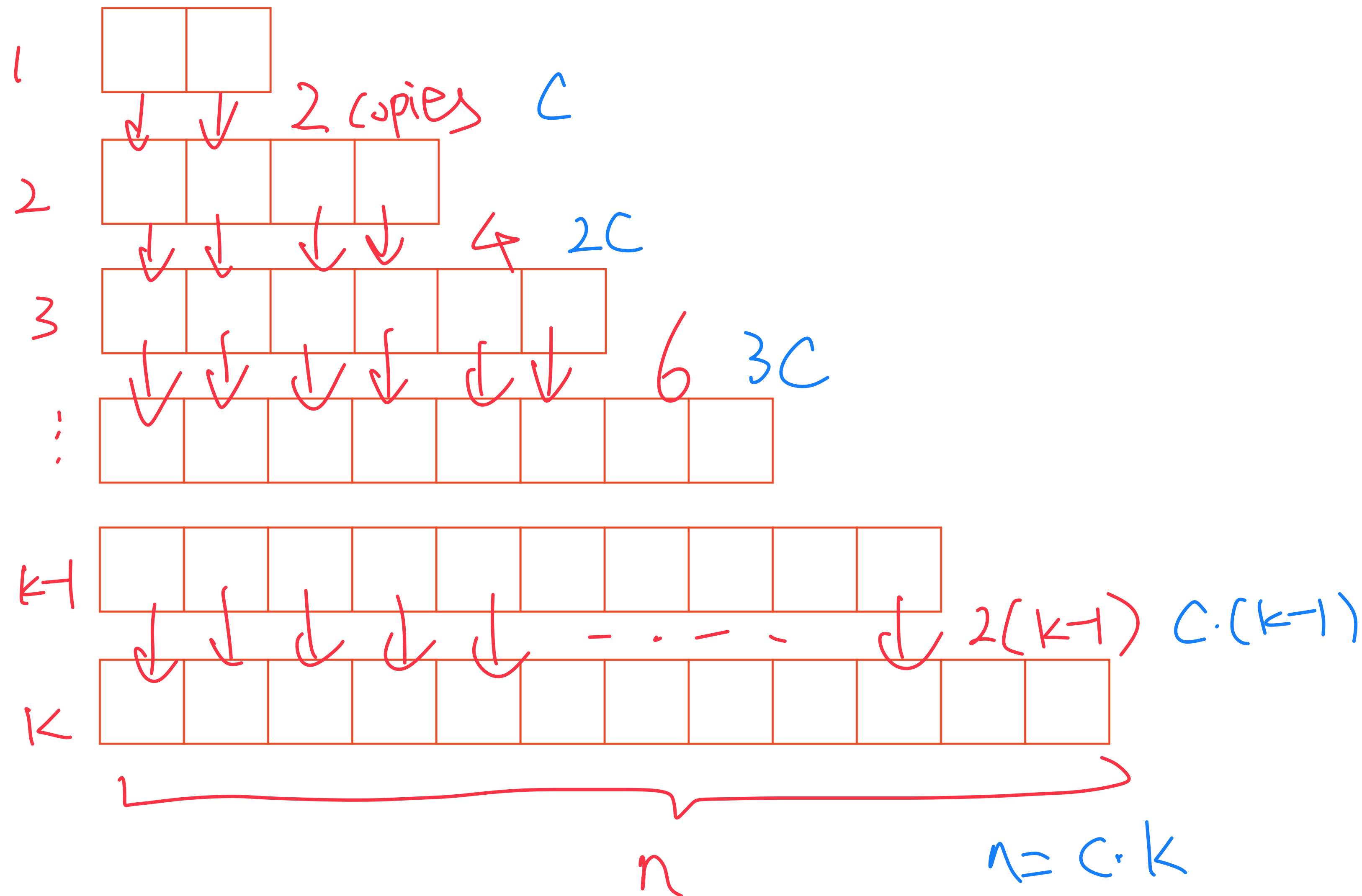


How much time will it
take to create elements
of size n ??

⇓
How many copies
I need to make??

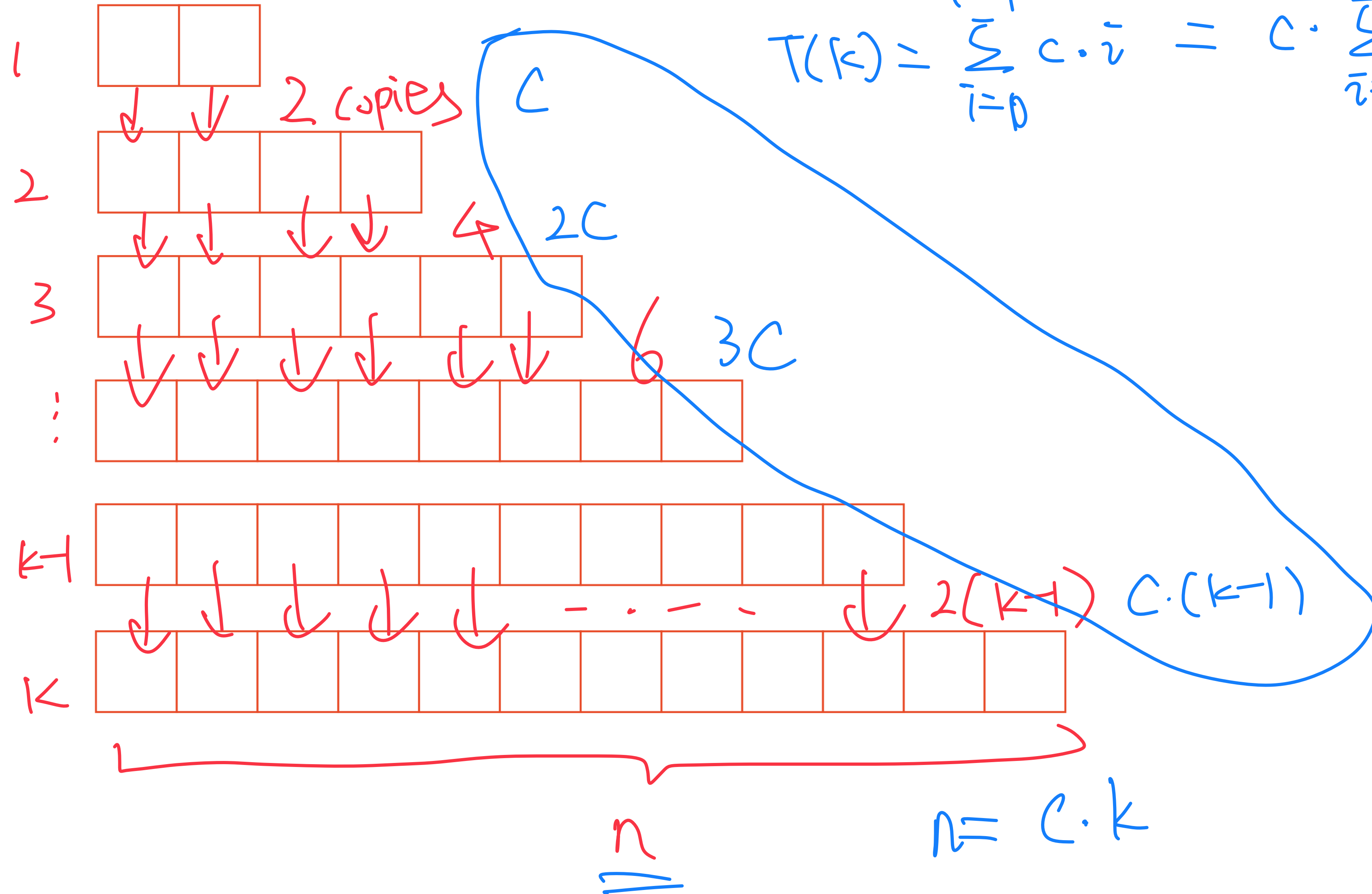
Resize Strategy – when full, add 2

k-levels



Resize Strategy – when full, add 2

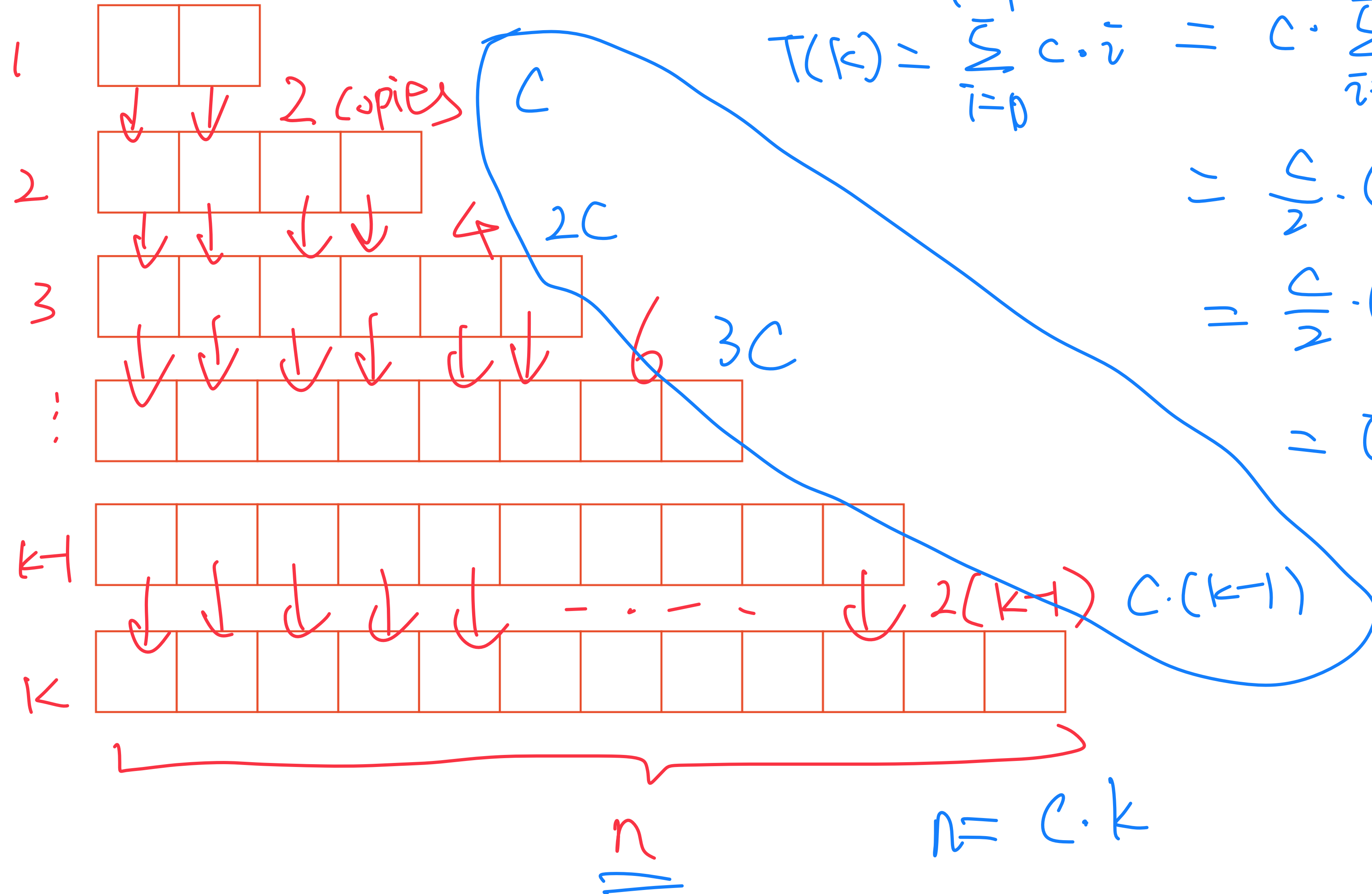
k-levels



$$T(k) = \sum_{i=0}^{k-1} c \cdot 2^i = c \cdot \sum_{i=0}^{k-1} 2^i = c \cdot \frac{(k-1) \cdot k}{2}$$

Resize Strategy – when full, add 2

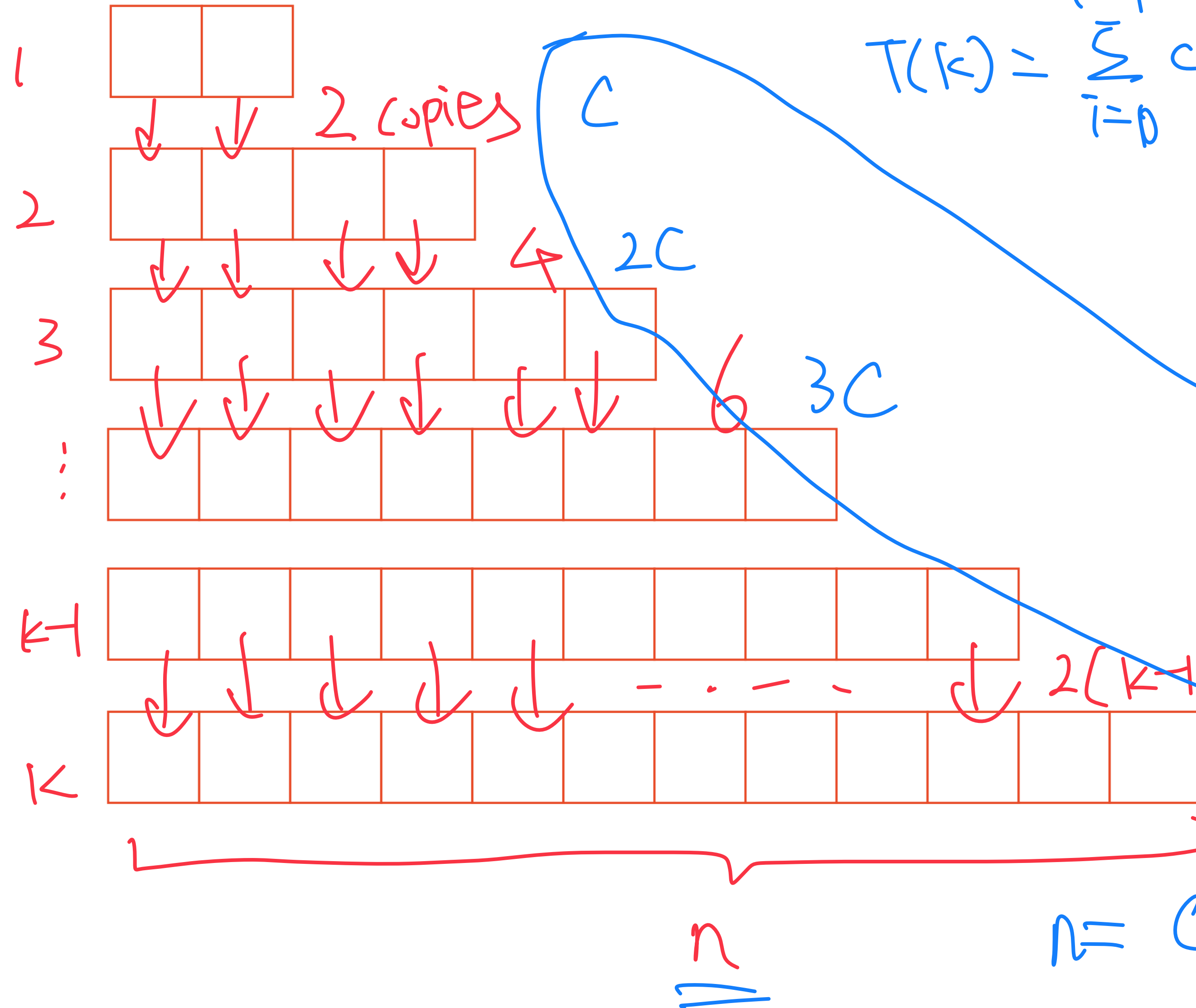
k-levels



$$\begin{aligned}
 T(k) &= \sum_{i=0}^{k-1} C \cdot i = C \cdot \sum_{i=0}^{k-1} i = C \cdot \frac{(k-1) \cdot k}{2} \\
 &= \frac{C}{2} \cdot (k-1) \cdot k \\
 &= \frac{C}{2} \cdot \left(\frac{n}{C} - 1\right) \cdot \frac{n}{C} \\
 &= O(n^2)
 \end{aligned}$$

Resize Strategy – when full, add 2

k-levels



$$T(k) = \sum_{i=0}^{k-1} c \cdot i = c \cdot \sum_{i=0}^{k-1} i = c \cdot \frac{(k-1) \cdot k}{2}$$

$$= \frac{c}{2} \cdot (k-1) \cdot k$$

$$= \frac{c}{2} \cdot \left(\frac{n}{c} - 1\right) \cdot \frac{n}{c}$$

$$= O(n^2)$$

For n elements

For every element:

$$\frac{O(n^2)}{n} = O(n)$$

Resize Strategy – when full, double the size

k

0



1



2



k-1

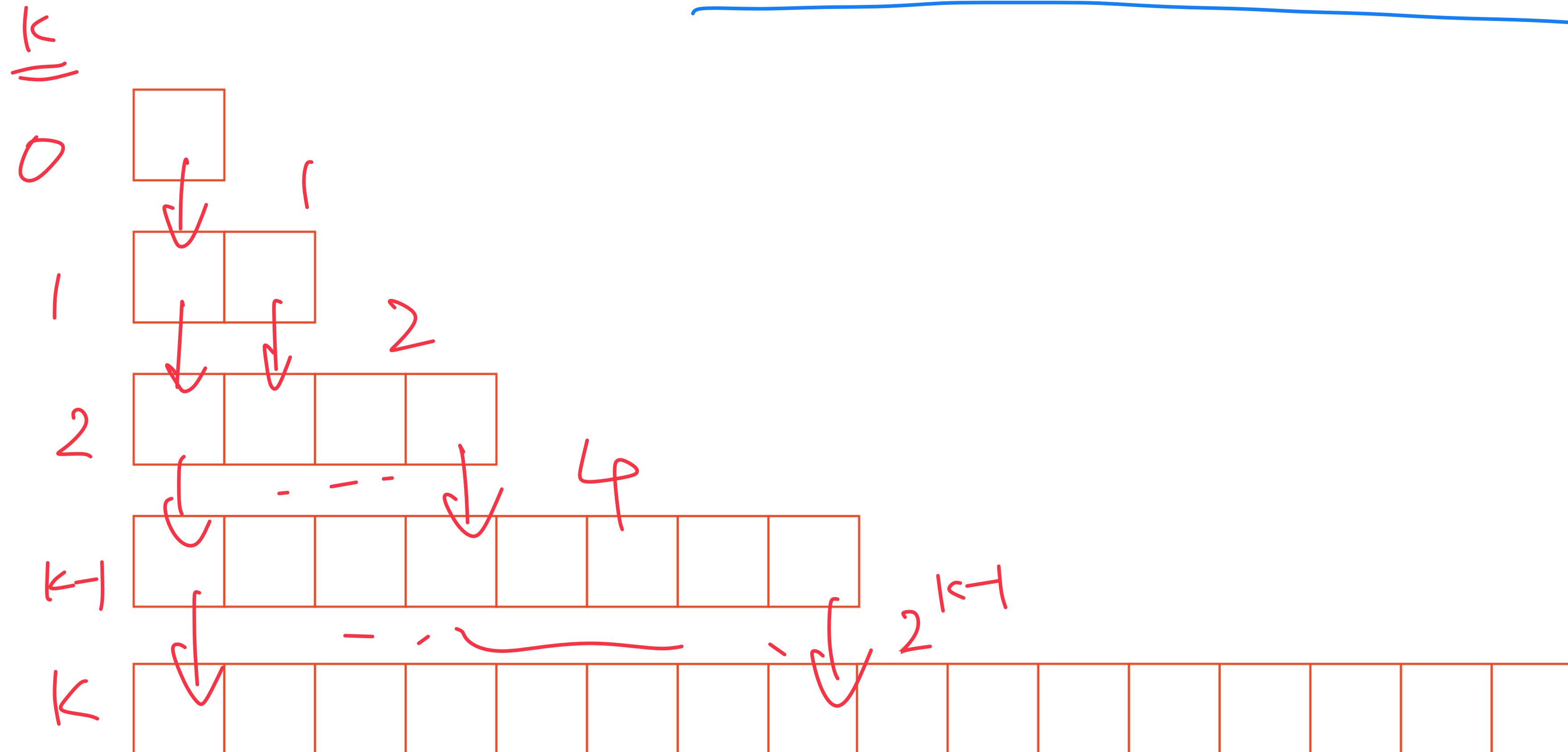


k



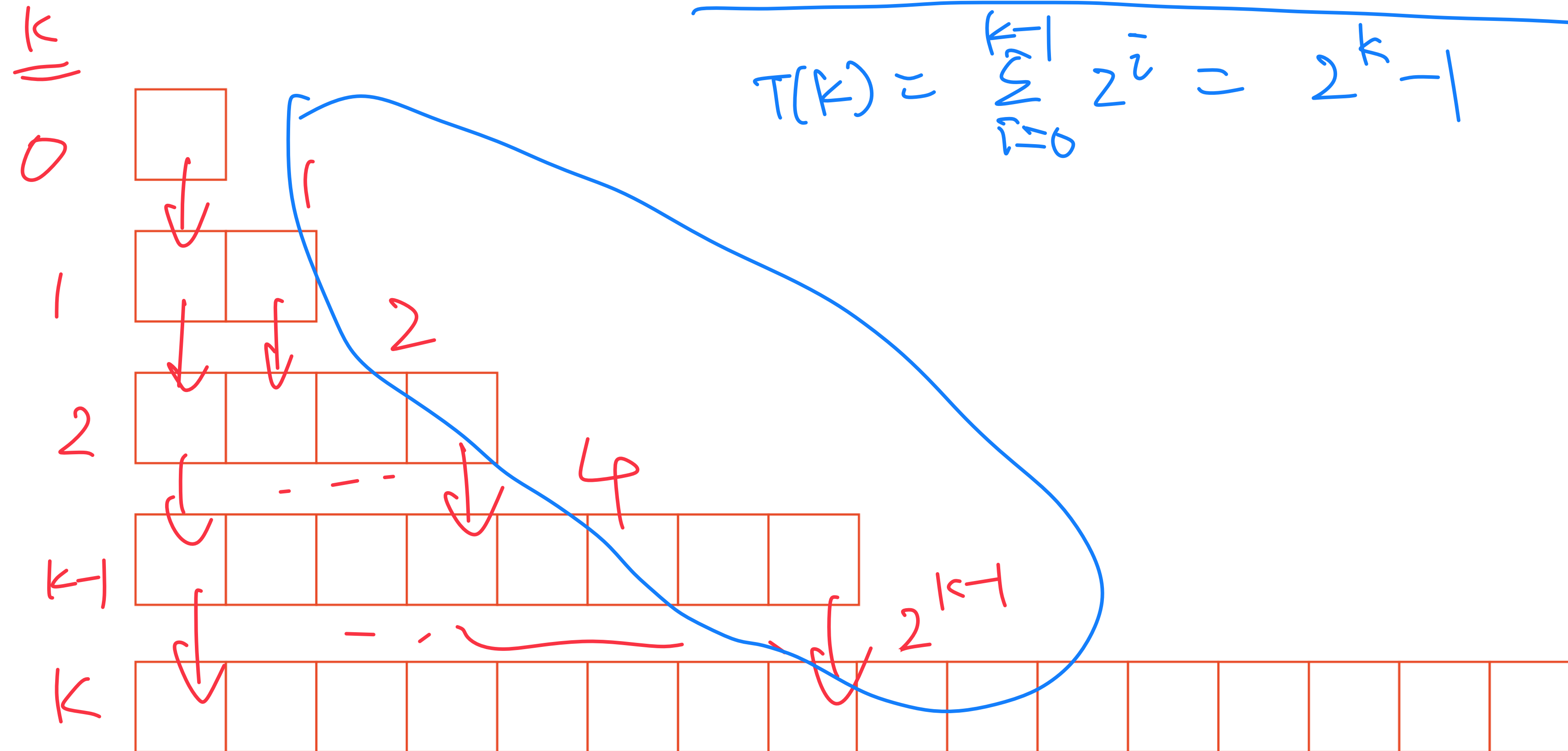
$$n = 2^k$$

Resize Strategy – when full, double the size



$$n = 2^k$$

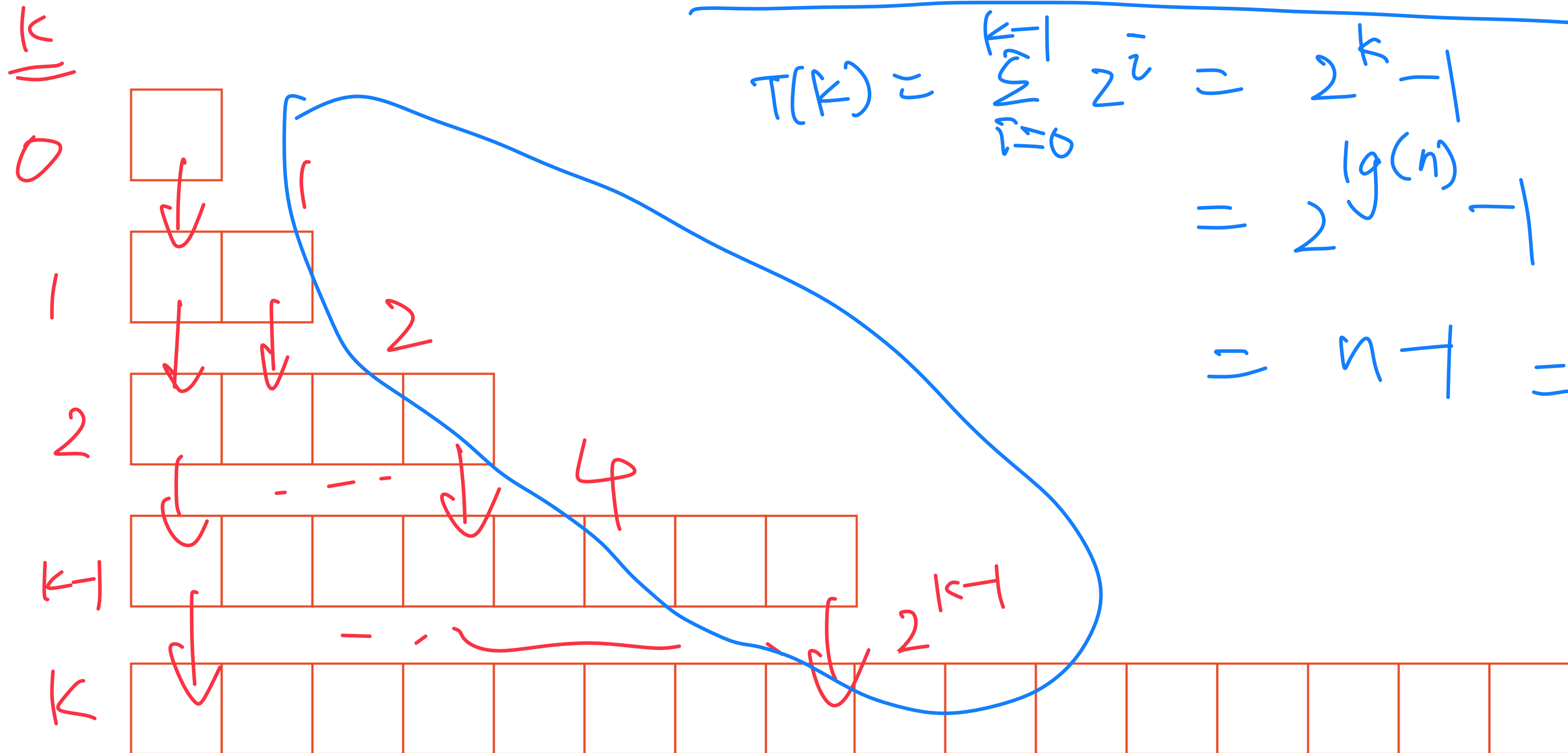
Resize Strategy – when full, double the size



$$n = 2^k$$

Resize Strategy – when full, double the size

$T(k) = \sum_{i=0}^{k-1} 2^i = 2^k - 1$
 $= 2^{\lg(n)} - 1$
 $= n - 1 = O(n)$



$$n = 2^k \Rightarrow k = \lg(n)$$

Resize Strategy – when full, double the size

$T(k) = \sum_{i=0}^{k-1} 2^i = 2^k - 1 = 2^{\lg(n)} - 1 = n - 1 = O(n)/n$

per insertion: $\frac{O(1)}{\text{Amortized}}$

constant insertion time

$n = 2^k \Rightarrow k = \lg(n)$

Implications of Design

Lots of ways to implement
→ difference!

1.

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

2.

```
T ** arr;
```

```
struct ListNode {  
    T * data;  
    ...  
};
```

3.

```
T * arr;
```

```
struct ListNode {  
    T data;  
    ...  
};
```

Implications of Design

Lots of ways to implement
→ difference!

1. Storage by reference

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

2. Storage by pointer

```
T ** arr;
```

```
struct ListNode {  
    T * data;  
    ...
```

3. Storage by value

```
T * arr;
```

```
struct ListNode {  
    T data;  
    ...
```


Implications of Design

Lots of ways to implement
→ difference!

1. Storage by reference

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

only possible with linked list.
can we have a dynamic array of ref ??

2. Storage by pointer

```
T ** arr;
```

```
struct ListNode {  
    T * data;  
    ...  
};
```

3. Storage by value

```
T * arr;
```

```
struct ListNode {  
    T data;  
    ...  
};
```

```
int a=0,b=0,c=0;  
int& arr[] = {a, b, c};
```

```
: error: declaration of 'arr' as array of references
```

There shall be no references to references, **no arrays of references**, and no pointers to references.

References are not objects. They do not occupy memory. — Declaring an array of nothing

Implications of Design

Lots of ways to implement
→ difference!

1. Storage by reference

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

only possible with linked list.
can we have a dynamic array of ref ??

2. Storage by pointer

```
T ** arr;
```

```
struct ListNode {  
    T * data;  
    ...  
};
```

3. Storage by value

```
T * arr;
```

```
struct ListNode {  
    T data;  
    ...  
};
```


Implications of Design

	Storage by Reference	Storage by Pointer	Storage by Value
Who manages the lifecycle of the data?			
Is it possible for the data structure to store NULL?			
If the data is manipulated by user code while in our data structure, is the change reflected in our data structure?			
Is it possible to store literals?			
Speed			

Data Lifecycle

if client delete data, will cause problem to me??

Storage by reference:

1	Sphere s;
2	myStack.push(s);

Storage by pointer:

1	Sphere s;
2	myStack.push(s);

Storage by value:

1	Sphere s;
2	myStack.push(s);

Data Lifecycle

Storage by reference:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

*if client delete data, will cause problem to me??
= Yes, data may not be there!
★ client code controls lifecycle!!*

Storage by pointer:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

Storage by value:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

Data Lifecycle

Storage by reference:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

if client delete data, will cause problem to me??
= Yes, data may not be there!
★ client code controls lifecycle!!

Storage by pointer:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

→ same as reference

Storage by value:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

Data Lifecycle

Storage by reference:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

if client delete data, will cause problem to me??
= Yes, data may not be there!
★ client code controls lifecycle!!

Storage by pointer:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

→ same as reference

Storage by value:

1	<code>Sphere s;</code>
2	<code>myStack.push(s);</code>

Copy data, our code controls

Possible to store NULL?

Storage by reference:

Can we have data structure if we store by reference $\Rightarrow$ store NULL?!

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

Storage by pointer:

```
T ** arr;
```

Storage by value:

```
T * arr;
```

Possible to store NULL?

Storage by reference:

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

Can we have data structure if we store by reference to store NULL??
No no address of nothing.
Non-existing memory.

Storage by pointer:

```
T ** arr;
```

Storage by value:

```
T * arr;
```

Possible to store NULL?

Storage by reference:

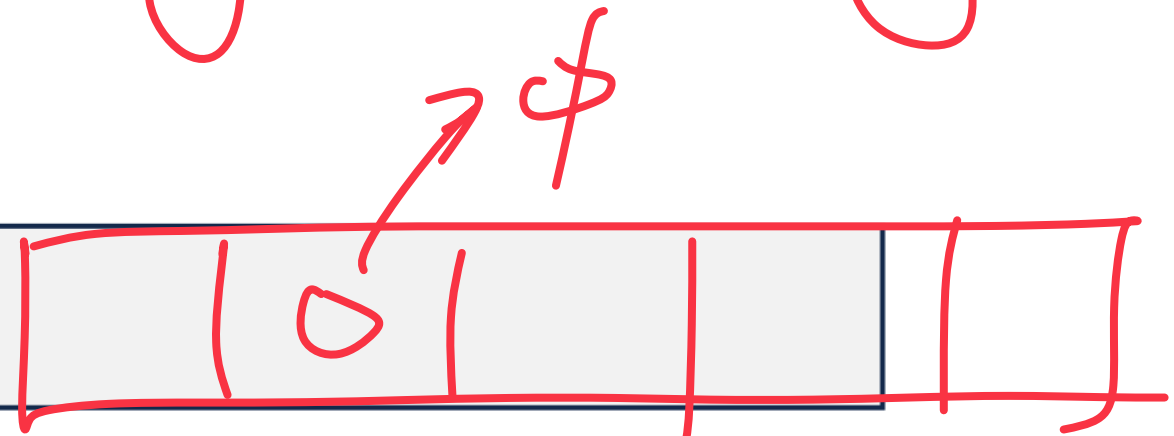
```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

Can we have data structure if we store by reference to store NULL??
No no address of nothing.
Non-existing memory.

Storage by pointer:

```
T ** arr;
```

Yes, NULL Pointer

A diagram showing a horizontal array of five boxes. The second box from the left contains the value '0'. A red arrow points from the top of this box to a red symbol that looks like a crossed circle (ϕ), representing a null pointer.

Storage by value:

```
T * arr;
```


Possible to store NULL?

Storage by reference:

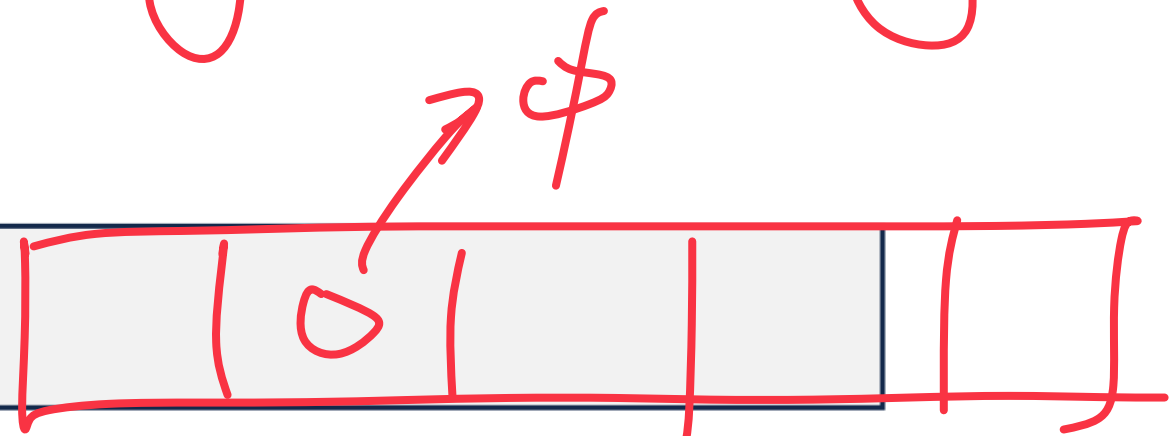
```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

Can we have data structure if we store by reference to store NULL??
No no address of nothing.
Non-existing memory.

Storage by pointer:

```
T ** arr;
```

Yes, NULL Pointer

A diagram showing a horizontal array of five memory cells. The second cell from the left contains the value '0'. A red arrow points from the top of this cell to a red symbol that looks like a crossed circle (ϕ), representing a null pointer.

Storage by value:

```
T * arr;
```

Sphere s(NULL)??

Possible to store NULL?

Storage by reference:

```
struct ListNode {  
    T & data;  
    ListNode * next;  
    ListNode(T & data) : data(data), next(NULL) { }  
};
```

Can we have data structure if we store by reference to store NULL??

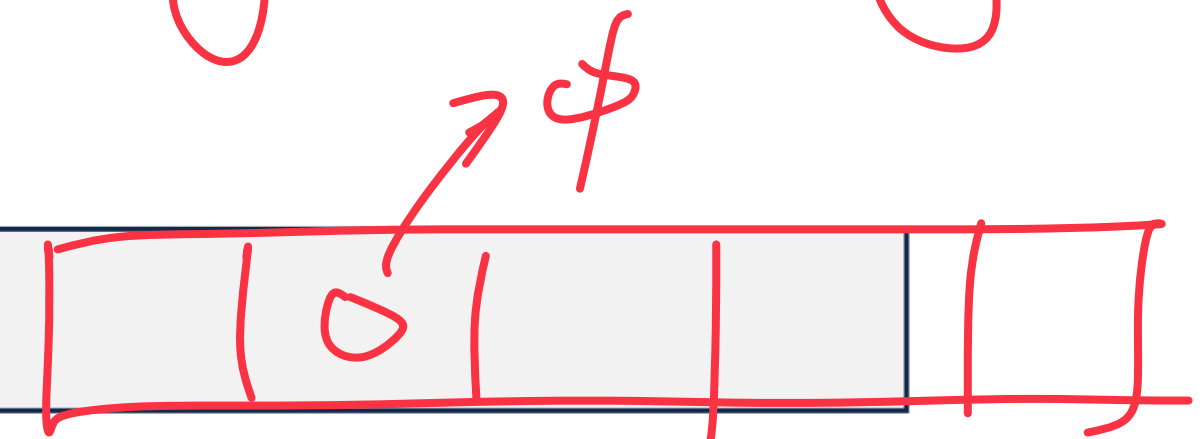
No no address of nothing.

Non-existing memory.

Storage by pointer:

```
T ** arr;
```

Yes, NULL Pointer



Storage by value:

```
T * arr;
```

Sphere s(NULL)??

No : if we have an instance, it cannot be NULL!

Data Modifications

```
1 Sphere s(1);  
2 myStack.push(s);  
3  
4 s.setRadius(42);  
5  
6 Sphere r = myStack.pop();  
7 // What is r's radius?
```

Ref

can
modify
??

Data Modifications

```
1 Sphere s(1);  
2 myStack.push(s);  
3  
4 s.setRadius(42);  
5  
6 Sphere r = myStack.pop();  
7 // What is r's radius?
```

Ref

Pointer

Yes,

(42)

can
modify
??

Data Modifications

```
1 Sphere s(1);  
2 myStack.push(s);  
3  
4 s.setRadius(42);  
5  
6 Sphere r = myStack.pop();  
7 // What is r's radius?
```

Ref

Yes,

(42)

Pointer

Yes

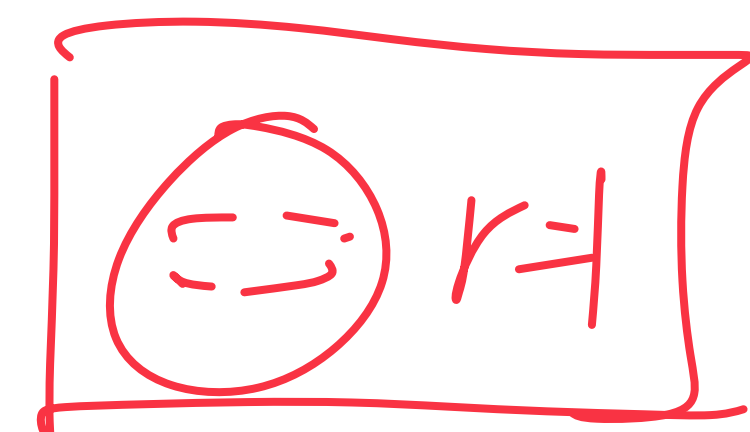
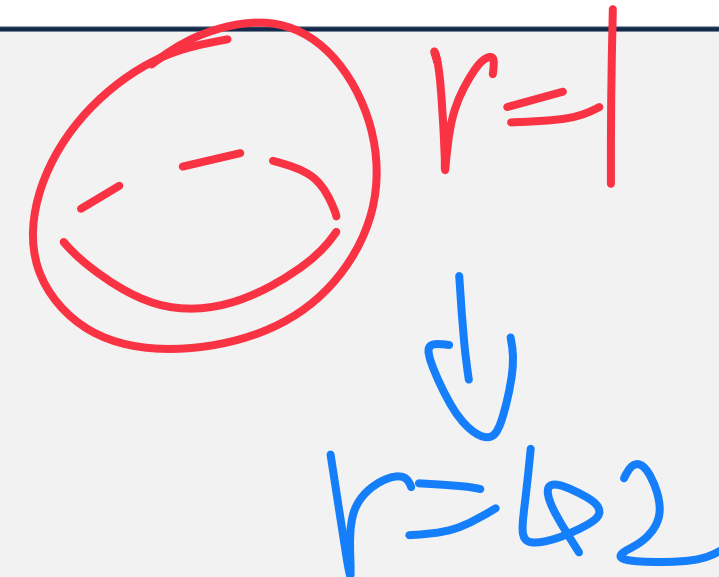
(42)

Value

can
modify
??

Data Modifications

```
1 Sphere s(1);  
2 myStack.push(s);  
3  
4 s.setRadius(42);  
5  
6 Sphere r = myStack.pop();  
7 // What is r's radius?
```



Ref

Yes,
(42)

Pointer

Yes
(42)

Value

No
X (1)

can
modify
??

Literals

- expression not stored in a variable

```
1 Stack<int> myStack;  
2  
3 myStack.push(1);  
4 myStack.push(2);  
5 myStack.push(3);
```

Ref

Pointer

X
there is no
memory

- compile error

Literals

- expression not stored in a variable

```
1 Stack<int> myStack;  
2  
3 myStack.push(1);  
4 myStack.push(2);  
5 myStack.push(3);
```

Ref

X
there is no
memory

- compile error

Pointer

X
Need to
have memory

Value

Literals

- expression not stored in a variable

```
1 Stack<int> myStack;  
2  
3 myStack.push(1);  
4 myStack.push(2);  
5 myStack.push(3);
```

Ref

X
there is no
memory

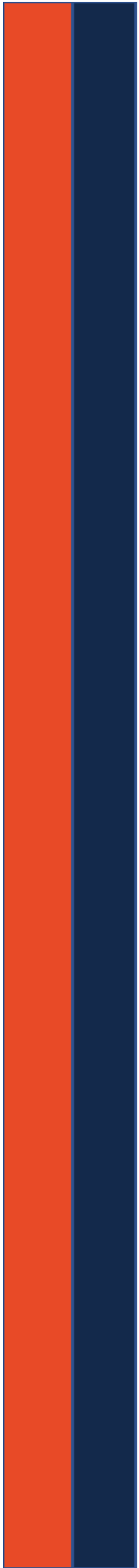
- compile error

Pointer

X
Need to
have memory

Value

✓
make a copy



Speed

ref

—

Fast

pointer

—

Fast

Value

—

Slow for large objects

Implications of Design

	Storage by Reference	Storage by Pointer	Storage by Value
Who manages the lifecycle of the data?	Client	Client	Our copy
Is it possible for the data structure to store NULL?	no	yes	no
If the data is manipulated by user code while in our data structure, is the change reflected in our data structure?	yes	yes	no
Is it possible to store literals?	no	no	yes
Speed	fast	fast	Slow on large objs

Queue.h

```
1 #ifndef QUEUE_H
2 #define QUEUE_H
3
4 template <class T>
5 class Queue {
6     public:
7
8
9
10
11
12
13
14
15
16
17     private:
18
19
20 };
21
22 #endif
```

CS 225 – Things To Be Doing

Quiz 2 is on Thursday (Lecture 1-9)

MP2 – Due Monday, March 16, 11:59pm

Lab: lab_gdb2 – Due Sunday, March 15, 11:59pm

One of the hardest labs of the semester, important to work with gdb